合肥工业大学

# 计算机组成原理

贾伟
计算机与信息学院



合肥工业大学计算机学院贾伟

# 人类社会发展总历程：以生产力为标准




合肥工业大学计算机学院贾伟

# 四次工业革命

**第一次工业革命**
- 机械化
- 标志：蒸汽机
- 机器取代人力
- 进化到工业

**第二次工业革命**
- 电气化
- 电力广泛应用
- 电力取代蒸汽
- 大规模工业

**第三次工业革命**
- 自动化
- 电子技术
- 信息技术
- 自动化生产

**第四次工业革命**
- 智能化
- 信息通讯技术
- 信息物理系统
- 智能制造

## 信息技术革命小周期

| 架构化 | 数字化 | 网络化 | 智能化 |
| --- | --- | --- | --- |
| 1950-1970 | 1970-1990 | 1990-2016 | 2016-Now |

合肥工业大学计算机学院贾伟

# 互联网发展的不同阶段

互联网诞生 → PC互联网 → 移动互联网 → 人工智能时代

1989 — 2010 — 2016 — Now

- 蒂姆·伯纳斯·李

发明万维网、第一个浏览器和使万维网得以扩展的基本协议和算法

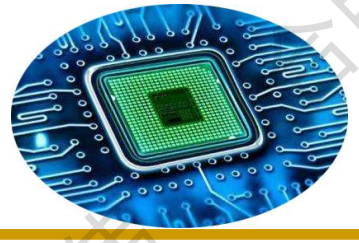






合肥工业大学计算机学院贾伟

# 新兴信息技术推动社会发展

| | | | |
|---|---|---|---|
| 大数据  | 云计算  | 社交关系网络  | 物联网  |
| 移动互联网  | 基于位置的服务  | 人工智能  | 移动通信  |
| 芯片  | 传感器  | 区块链  | 元宇宙  |

合肥工业大学计算机学院贾伟

# 第三次浪潮

## 信息革命

## 第四次工业革命

人类社会正处在新的社会形态变革中

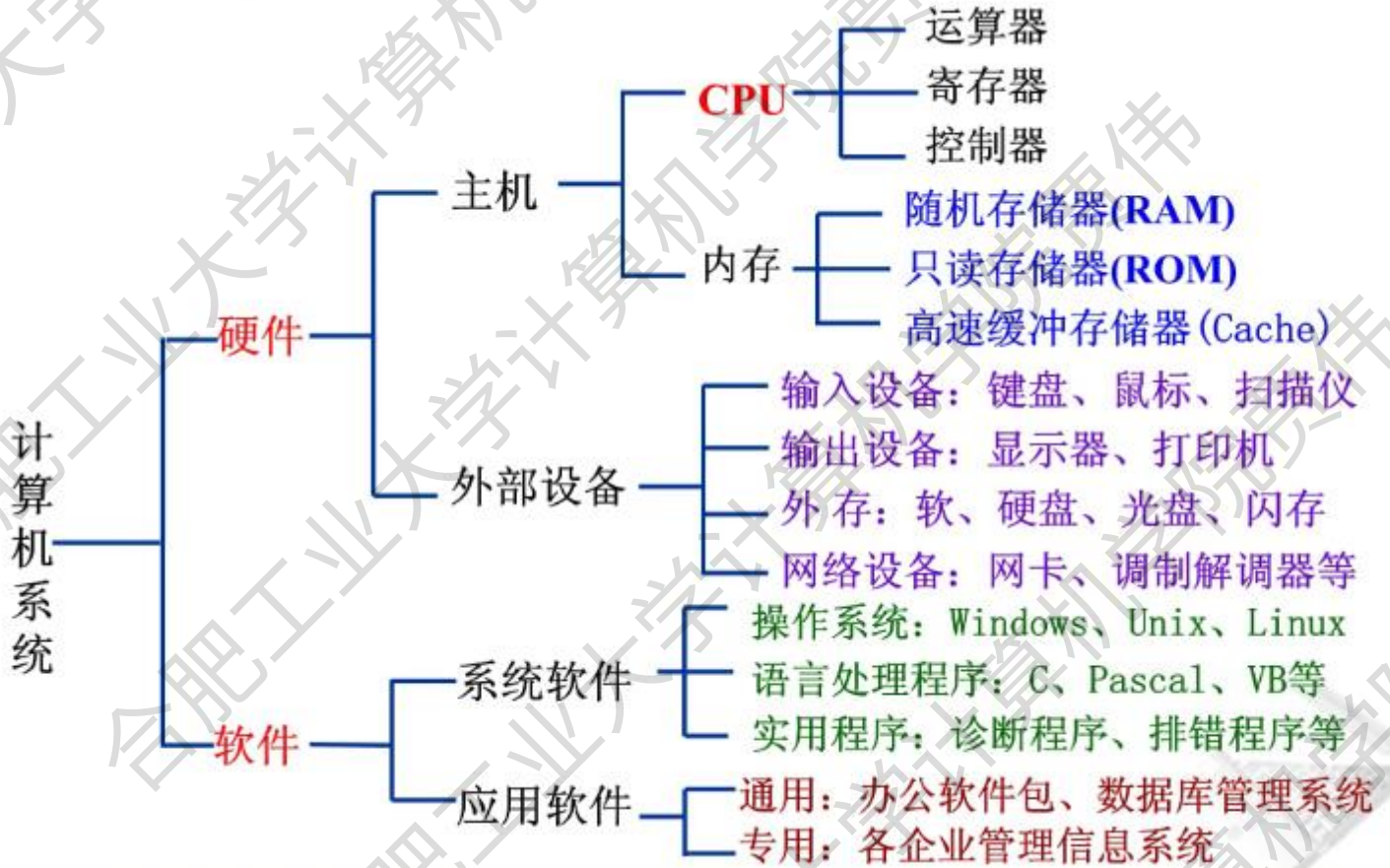
人类正处于进化的新起点

旧有的秩序、社会关系、社会模式被打破、推翻、替代

**重塑、构建**新的秩序、社会关系、社会模式

合肥工业大学计算机学院贾伟

**计算机硬件**是计算机系统中所有物理设备的总称，是看得见、摸得着的实体部件。它通过电子、机械或光电等方式直接执行计算、存储、输入和输出等操作，是计算机实现功能的物质基础。




**计算机软件**是计算机中程序、数据和相关文档的集合，是一种逻辑实体而非物理设备。它通过指令控制硬件资源，实现特定功能或解决具体问题，是用户与硬件之间的桥梁。

## 一、硬件系统（物理实体）

### 1.核心部件

1. **CPU（中央处理器）**：运算器+控制器，执行指令
2. **存储器**
    1. RAM（随机存取内存）：临时存储运行程序与数据（如智能算法的实时计算）
    2. ROM（只读存储器）：固件存储（如机器人控制系统的启动程序）
    3. 硬盘/SSD：长期存储操作系统与文件（如自动驾驶车辆的数据日志）

### 2.输入/输出设备

1. **输入**：键盘、摄像头、传感器（如工业机器人的视觉采集模块）
2. **输出**：显示器、打印机、执行器（如智能音箱的语音合成模块）

### 3.主板与外设

1. 主板：连接CPU、内存、硬盘等核心组件
2. 总线：数据传输通道（如PCIe接口用于高速数据交换）
3. 电源：为系统供电（如高精度服务器电源保障AI算力稳定性）

合肥工业大学计算机学院贾伟

## 二、软件系统（逻辑与程序）

### 1.操作系统（OS）

1. 管理硬件资源（如Linux驱动工业机器人运动控制）
2. 提供用户接口（如Windows系统支撑AI开发环境）

### 2.系统软件

1. 编译器/解释器：将代码转化为机器指令（如PyTorch框架加速深度学习模型训练）
2. 数据库管理系统：高效存储与检索结构化信息（如智能电网的数据分析）

### 3.应用软件

1. **通用工具**：浏览器、办公软件（如Office 365支持远程协作开发）
2. **专业领域**：
    1. CAD设计软件（如智能汽车的三维建模）
    2. AI应用（如商汤科技的计算机视觉算法平台）

合肥工业大学计算机学院贾伟

## 三、支撑环境（现代扩展）

### 1.云计算与边缘计算

1. 云端：提供弹性算力（如AWS GPU实例训练大规模语言模型）
2. 边缘端：本地化实时处理（如自动驾驶汽车的毫秒级决策）

### 2.网络与安全

1. 协议栈：TCP/IP保障数据传输（如工业互联网的安全通信）。
2. 加密技术：保护AI模型与用户隐私（如联邦学习中的数据脱敏）。

合肥工业大学计算机学院贾伟

**五大核心硬件组件**

**1.中央处理器（CPU - Central Processing Unit）**

1. **功能**：计算机的“大脑”，负责执行指令、处理数据和控制其他部件。
2. **组成**：
    1. **运算器（ALU）**：执行算术（加减乘除）和逻辑（与或非）运算。
    2. **控制器（CU）**：从内存读取指令，解码并协调各部件工作。
    3. **寄存器**：高速临时存储单元，存放当前执行的指令、数据或地址。
3. **关键指标**：主频（GHz）、核心数（单核/多核）、缓存容量（L1/L2/L3）。

**2.存储器（Memory）**

1. **主存储器（内存 - RAM/ROM）**
    1. **RAM（随机存取存储器）**：临时存储运行中的程序和数据，断电后数据丢失。
    2. **ROM（只读存储器）**：存储固件（如BIOS），断电后数据保留。
2. **辅助存储器（外存）**：
    1. **硬盘（HDD/SSD）**：长期存储操作系统、软件和文件。
    2. **闪存（U盘、SD卡）**：便携式存储设备。
3. **高速缓存（Cache）**：位于CPU与内存之间，加速数据访问。

合肥工业大学计算机学院贾伟

## 五大核心硬件组件

### 3.输入设备（Input Devices）

1. **功能**：将用户指令或外部数据转换为计算机可处理的信号。
2. **常见设备**：键盘、鼠标、触摸屏、麦克风、摄像头、扫描仪等。

### 4. 输出设备（Output Devices）

1. **功能**：将计算机处理结果转换为人类可感知的形式。
2. **常见设备**：显示器、打印机、扬声器、投影仪等。

### 5. 系统总线（Bus）

1. **功能**：连接各硬件组件，传输数据、地址和控制信号。
2. **分类**：
    1. **数据总线**：双向传输数据（位数决定字长，如64位）。
    2. **地址总线**：单向传输内存或设备的地址（位数决定寻址空间）。
    3. **控制总线**：传输控制信号（如读/写、中断请求）。

合肥工业大学计算机学院贾伟

**硬件协同工作流程**

**1.指令执行周期**

1. **取指令**：CPU从内存读取指令。
2. **解码指令**：控制器解析指令含义。
3. **执行指令**：运算器处理数据，结果存回寄存器或内存。
4. **重复循环**：周而复始执行程序指令。

**2.输入输出流程（I/O）**

1. 输入设备 → 内存 → CPU处理 → 内存 → 输出设备。
2. 通过**I/O接口**（如USB、HDMI）连接外设，由**DMA控制器**直接管理数据传输，减少CPU负担。

合肥工业大学计算机学院贾伟

**扩展硬件（现代计算机常见组件）**

**1.图形处理器（GPU）**

1. 专为并行计算设计，加速图形渲染和机器学习任务。

**2.主板（Motherboard）**

1. 集成CPU插槽、内存插槽、扩展槽（PCIe）、芯片组（北桥/南桥）等，负责协调各部件通信。

**3.电源与散热系统**

1. **电源**：将交流电转换为直流电，为硬件供电。
2. **散热器/风扇**：防止CPU、GPU等部件过热。

**4.网络与通信模块**

1. 网卡、Wi-Fi模块、蓝牙芯片等，支持网络连接和数据传输。

合肥工业大学计算机学院贾伟

**硬件系统分层结构**

1.**底层硬件**：晶体管、逻辑门、电路板。

2.**功能模块**：CPU、存储器、I/O设备。

3.**系统级互联**：总线、主板、接口协议。

**重要概念**

•**冯•诺依曼体系结构**：现代计算机的基础，核心是“存储程序”思想（程序和数据共同存储于内存）。

•**哈佛架构**：程序存储与数据存储分离，用于嵌入式系统（如单片机）。

•**摩尔定律**：集成电路上可容纳的晶体管数量每18-24个月翻倍（现已趋缓）。

合肥工业大学计算机学院贾伟

## 一、软件系统的分类

计算机软件系统通常分为两大类：**系统软件**和**应用软件**。

1.**系统软件**

1. **功能**：管理硬件资源，为应用软件提供运行环境。
2. **主要类型**：
    1. **操作系统（OS）**：如Windows、Linux、macOS，负责资源分配、进程调度、文件管理等。
    2. **驱动程序**：控制特定硬件设备（如显卡、打印机）。
    3. **编译器/解释器**：将高级语言（如C、Python）转换为机器可执行的代码。
    4. **实用工具**：如磁盘清理工具、防病毒软件、调试工具。

2.**应用软件**

1. **功能**：解决用户的具体需求。
2. **常见类型**：
    1. 办公软件（如Office、WPS）、浏览器（Chrome、Firefox）、游戏、数据库管理系统（MySQL）等。

合肥工业大学计算机学院贾伟

## 二、操作系统的核心功能

操作系统是软件系统的核心，直接与硬件交互，主要功能包括：

1.**进程管理**

1. 创建、调度和终止进程，实现多任务并行（如时间片轮转、优先级调度）。
2. 通过**进程控制块（PCB）**记录进程状态（运行、就绪、阻塞）。

2.**内存管理**

1. 分配和回收内存空间，实现虚拟内存（通过分页/分段技术扩展物理内存限制）。
2. 保护不同进程的内存空间，防止越界访问。

3.**文件系统**

1. 管理磁盘上的文件存储（如FAT32、NTFS、ext4）。
2. 提供文件读写接口，支持目录树结构和权限控制。

4.**设备管理**

1. 通过驱动程序控制I/O设备（如键盘、显示器），处理中断请求（IRQ）。
2. 提供统一的设备访问接口（如POSIX标准）。

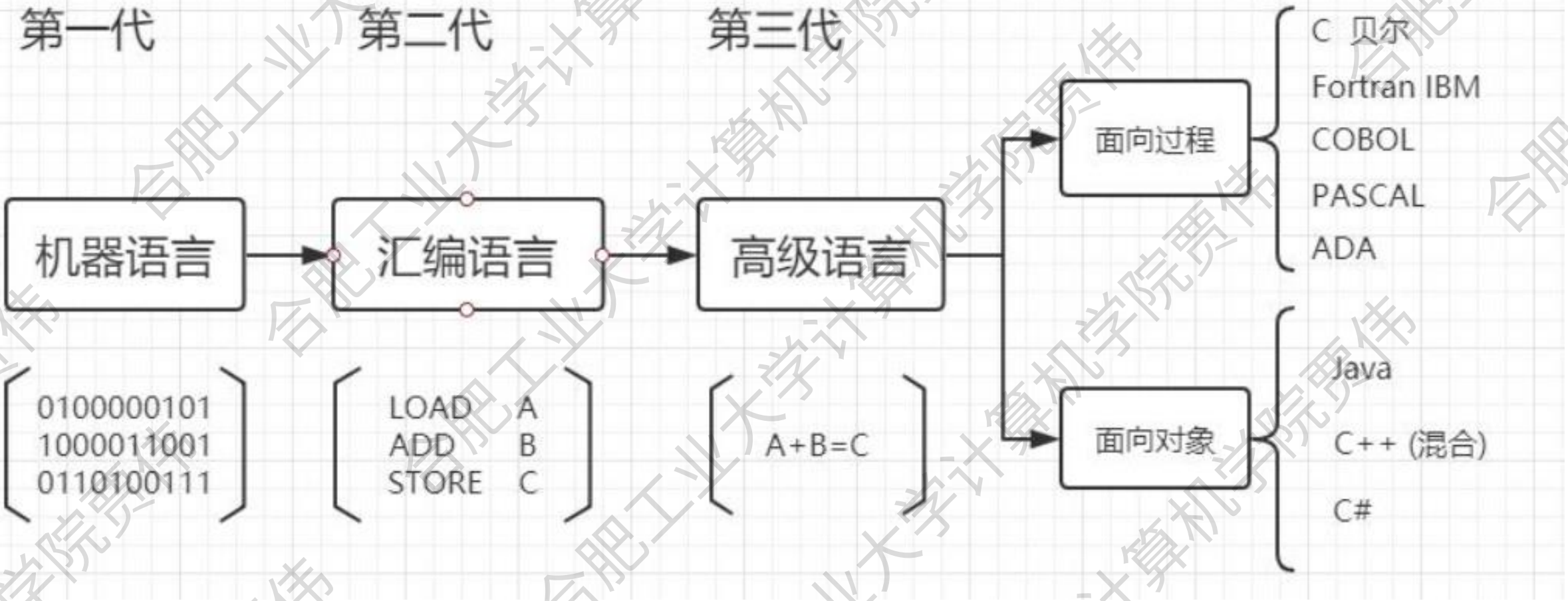
5.**用户接口**

1. **命令行界面（CLI）**：如Linux的Bash、Windows的PowerShell。
2. **图形用户界面（GUI）**：如Windows的桌面环境、macOS的Finder。

合肥工业大学计算机学院贾伟

# 第一章：计算机系统概论：计算机语言






合肥工业大学计算机学院贾伟

# 第一章：计算机系统概论：计算机语言



**低级语言时代**

- 低级语言：机器语言、汇编语言
- （1946 - 1953）

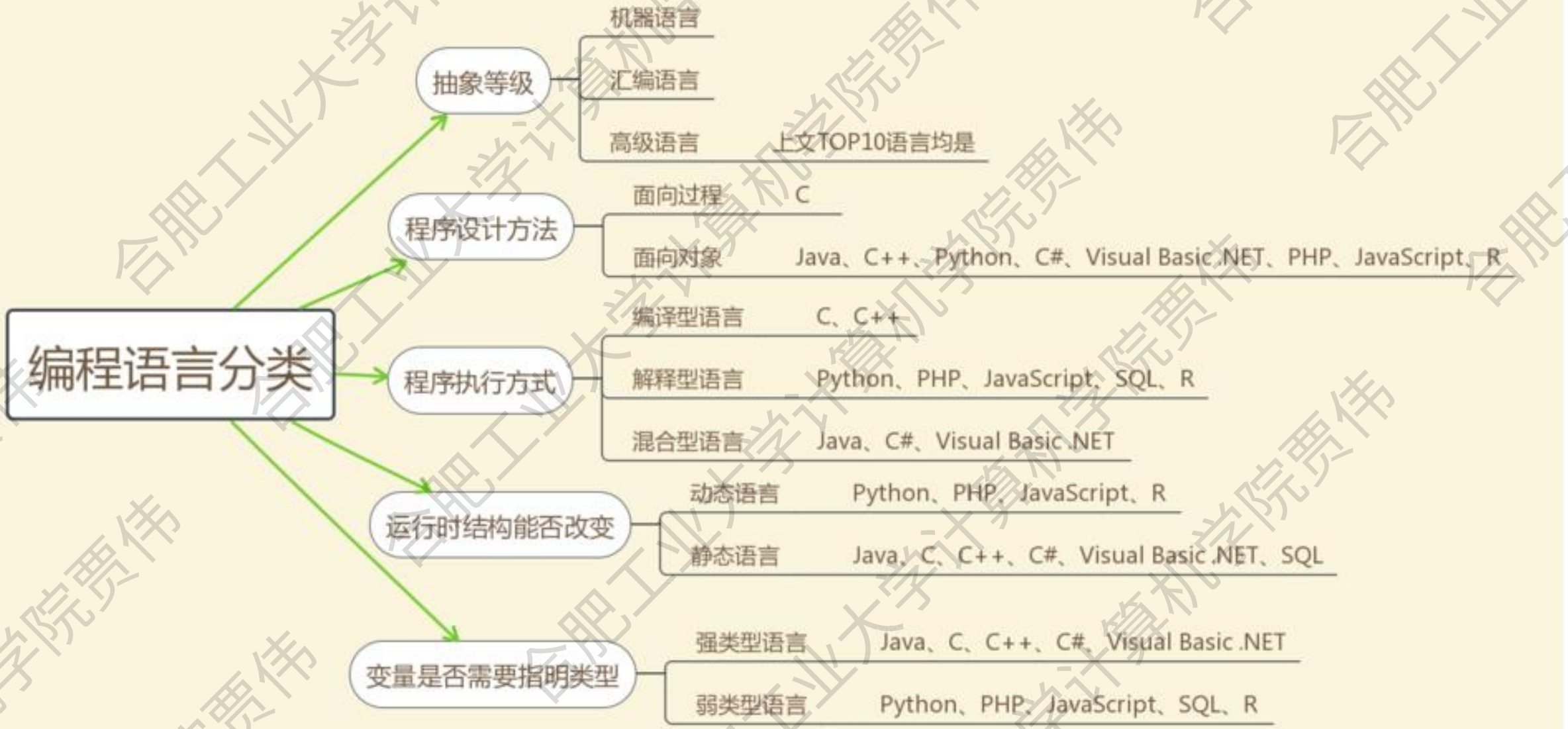
**高级语言时代**

- 现代程序语言革命的起点：C、C++
- （1954 - 至今）

**面向对象时代**

- 面向对象程序设计（OOP）：Java、Python、C++
- （90年代初 - 至今）

合肥工业大学计算机学院贾伟




合肥工业大学计算机学院贾伟

# 二、计算机系统的层次结构

计算机系统存在着层次结构，即计算机系统具有层次性，它由多级层次结构组成。

从功能上看，现代计算机系统可分为五个层次级别。



# 三、 计算机组成和计算机体系结构

**计算机体系结构:**是指那些能够被程序员所见到的计算机系统的属性，即概念性的结构与功能特性，通常是指用机器语言编程的程序员（也包括汇编语言程序设计者和汇编程序设计者）所看到的传统机器的属性，大都属于抽象的属性。由于计算机系统具有多级层次结构，因此，站在不同层次上编程的程序员所看到的计算机属性也是各不相同的。

**计算机体系结构**关注的是计算机系统的逻辑结构，包括指令集设计、数据类型、寄存器模型、内存访问模式等，它定义了程序员与硬件之间的界面。

**计算机组成原理：**是指如何实现计算机体系结构所体现的属性，它包含了许多对程序员来说是透明的（即程序员不知道的）硬件细节。

**计算机组成原理：**主要指的是计算机硬件组件之间的关系以及这些组件如何协同工作来实现计算机功能的科学，涉及的是计算机系统的物理实现、数据流和控制流

细节上，在体系结构涵盖的指令集架构等抽象层面，计算机组成原理会关心这些抽象如何通过具体的硬件电路、时序信号等实现。

# 三、 计算机组成和计算机体系结构

**COMPUTER ARCHITECTURE(计算机体系结构)**
计算机体系结构是计算机科学中对系统功能、组织和实现的一般描述。它作为程序员与系统硬件之间的中介，界定了功能和设计各种面向性能的衡量标准。

**功能性视角**
计算机体系结构首要确定计算机的功能。这包括了指令集的设计、数据类型的定义、寄存器和内存的访问方式。通过这些抽象，体系结构师决定程序员与系统之间的交互方式。**指令集**代表能被机器理解及执行的指令类型。这里，指令集的选择对系统的性能、能效、编写程序的难度都有深远影响。

**性能视角**
体系结构设计时，评估系统性能是一个核心指标。这涉及运算速度、处理能力、输入输出效率等。为了优化性能，体系结构师可能会设计**流水线**和采用**并行处理**机制。这些方法能提高执行指令的速度，并充分利用硬件资源。

# 三、 计算机组成和计算机体系结构

**COMPUTER ORGANIZATION(计算机组成原理)**
计算机组成原理涉及计算机系统的实际物理结构和实现方式。它关注的是硬件组件的设计与功能，以及这些组件如何集成为一个工作的全系统。

**硬件组件**
计算机组成原理围绕如处理器、内存、输入/输出设备等核心硬件组件。每个组件的设计都需要精密计算，确保它们能高效地执行针对特定任务的操作。例如，**处理器设计**涉及对算术逻辑单元（ALU）、寄存器、缓存和控制单元的详细布局。

**系统集成**
处理器、内存等组件需要通过特定的方式集成在一起，以便它们能协同工作。这通常需要**总线和接口**的深入理解，它们是各种硬件组件连接沟通的纽带。集成的流程包括对**地址映射**、**数据传输**和**信号同步**的研究。

合肥工业大学计算机学院贾伟

# 三、 计算机组成和计算机体系结构

**KEY DIFFERENCES(关键不同点)**
虽然计算机体系结构和组成原理有相交的地方，但是两者之间存在一些关键的差异。

**设计层次**
计算机体系结构更多地处于设计的逻辑层级，关注软硬件之间的接口。相比之下，**计算机组成原理**更偏向于物理实现，涉及具体的硬件设计、电路与信号处理等。

**抽象程度**
计算机体系结构涉及的指令集和编程模型等，处于较高的抽象层次。而组成原理则涉及底层的硬件电路和微观行为，如电子信号如何在处理器内传导。

合肥工业大学计算机学院贾伟

# 三、 计算机组成和计算机体系结构

**RELATIONSHIP AND INTERACTION(关系与交互)**
尽管存在区别，计算机体系结构和组成原理相互关联，共同影响一个系统的设计和性能。

**体系结构对组成原理的影响**
体系结构决定的设计选择会对组成原理面临的设计挑战产生影响。例如，采用的指令集决定了处理器需要支持的操作类型与实现复杂度。

**组成原理回馈体系结构**
当体系结构设计需要逐步实施时，可能会因为组成原理中的硬件装置的限制而进行调整。精简指令集（RISC）和复杂指令集（CISC）的争论就是体系结构受到组成原理实现约束的一个典型例子。

合肥工业大学计算机学院贾伟

# 三、 计算机组成和计算机体系结构

**EFFECTS ON SOFTWARE(对软件的影响)**
不同的体系结构和硬件实现对软件设计和优化也有所影响。

**软件兼容性**
软件开发必须考虑到体系结构定义的指令集。不同架构对应的编程模型和指令集会影响软件的移植性。

**软件性能优化**
了解硬件的具体实现，能够帮助开发者编写更有效利用系统资源的程序。例如，**缓存优化**和**并行计算**知识有助于改善软件运行时的性能表现。

合肥工业大学计算机学院贾伟

# 三、 计算机组成和计算机体系结构

**1. 什么是计算机组成原理和体系结构的区别?**

计算机组成原理和体系结构是计算机科学领域中两个重要的概念，虽然它们有关联，但是还是存在一些明显的区别。

**计算机组成原理**是指计算机由硬件和软件两个组成部分构成的基本原理和规则。它主要关注计算机的内部结构，包括处理器、存储器、输入输出设备等硬件组件的设计和工作原理，以及操作系统和编译器等软件系统的功能和运行机制。计算机组成原理研究的是计算机如何运行，以及如何利用硬件和软件资源进行数据处理和运算。

**计算机体系结构**则是指计算机系统从整体上组织和设计的原则和规范。它关注的是计算机硬件与软件之间的接口和交互，以及计算机系统的层次结构。计算机体系结构更注重于定义和描述计算机系统的层次和组织方式，以及不同层次之间的数据传递和通信方式。它不仅定义了计算机硬件和软件之间的接口规范，还规定了计算机系统的功能和性能要求，以及整体设计原则。

总结起来，计算机组成原理是关注计算机内部组件和运行原理的科学，而计算机体系结构则是关注计算机整体结构和组织方式的工程。计算机组成原理与体系结构相互影响，相互支持，共同构成了计算机科学的基础。

# 三、 计算机组成和计算机体系结构

**计算机组成原理与计算机体系结构的重要性是什么?**
计算机组成原理和计算机体系结构在计算机科学和信息技术领域具有重要的意义和作用。

计算机组成原理是计算机科学的基础，它研究计算机的内部构造、工作原理和功能实现方法。了解计算机组成原理可以帮助人们更好地理解计算机硬件和软件之间的关系，从而更好地设计和优化计算机系统。它为计算机科学的理论研究和技术发展提供了基础和指导。

计算机体系结构是计算机工程的重要组成部分，它规定了计算机硬件和软件之间的接口和交互方式，定义了计算机系统的层次结构和组织方式。良好的计算机体系结构可以提高计算机系统的性能和可靠性，减少系统的复杂性和成本。它为计算机工程师提供了设计、实现和维护计算机系统的准则和方法。

计算机组成原理和计算机体系结构的研究和应用对于推动计算机科学和信息技术的发展具有重要意义。它们为新的计算机体系结构和技术的研究和创新提供了基础和动力，促进了计算机科学和信息技术领域的发展和进步。

# 三、 计算机组成和计算机体系结构

**计算机组成原理和计算机体系结构有哪些应用领域?**

计算机组成原理和计算机体系结构的研究和应用涉及多个领域，对现代社会的各个行业和领域都有重要的影响。

在计算机科学领域，计算机组成原理和计算机体系结构的研究和发展为计算机系统的设计和优化提供了重要的理论基础和工程方法。它们被广泛应用于计算机硬件与软件系统的开发、优化和测试。

在信息技术行业，计算机组成原理和计算机体系结构的知识和技术是设计和开发高性能计算机系统和服务器的基础。它们被应用于云计算平台、大数据分析系统、人工智能算法等领域，为信息技术行业的发展提供强有力的支持。

在通信和网络行业，计算机组成原理和计算机体系结构被应用于网络设备和通信协议的设计和优化。它们为网络通信的可靠性、安全性和性能提供了保障，促进了通信和网络行业的发展。

在科学研究领域，计算机组成原理和计算机体系结构被广泛用于模拟和计算各种科学问题。它们可以帮助科学家们快速、准确地进行大规模数据处理、数值模拟和科学计算，推动了科学研究的进展。

总之，计算机组成原理和计算机体系结构在多个领域具有广泛的应用，对现代社会的信息化和科学技术的发展起到了重要的推动作用。

## 冯•诺依曼（John von Neumann）简介

### 生平概览

全名：约翰·冯·诺依曼（John von Neumann，1903–1957）
国籍：匈牙利裔美国科学家
领域：数学、计算机科学、物理学、经济学、博弈论
称号：“现代计算机之父”“博弈论之父”“量子力学的数学基础奠基人”

### 关键成就与贡献

**计算机科学**

冯·诺依曼架构：提出“存储程序”概念，奠定现代通用计算机设计基础（见EDVAC报告）。
计算机理论：推动二进制逻辑、程序自动执行等核心思想。

**数学与物理学**

量子力学：建立量子力学的严格数学框架（《量子力学的数学基础》）。
算子理论：在泛函分析中开创冯·诺依曼代数。
博弈论：与摩根斯特恩合著《博弈论与经济行为》，奠定现代博弈论基础。

**核武器与军事科学**

参与曼哈顿计划，设计原子弹内爆式结构。
推动氢弹研发，提出“辐射内爆”关键理论。

**经济学与计算**

提出“自复制自动机”理论（现代人工智能与生命科学的前瞻思想）。
研究经济系统的数学建模与计算机模拟。

**教育背景与职业生涯**

早年：

18岁发表首篇数学论文，精通6门语言。

获瑞士苏黎世联邦理工学院化学工程学位，同时攻读布达佩斯大学数学博士。

学术生涯：

普林斯顿高等研究院首批终身教授之一（与爱因斯坦、哥德尔共事）。

任教于柏林大学、汉堡大学，后移居美国。

**影响与遗产**

计算机革命：冯·诺依曼架构至今仍是计算机设计的核心范式。

跨学科先驱：其思想深刻影响数学、物理、经济学、人工智能等多个领域。

名言：

“若人们不相信数学简单，只因他们未意识到生命之复杂。”

“技术进步的终点是模拟一切可模拟之物。”

冯·诺依曼是20世纪最伟大的科学全才之一，其思想跨越学科边界，重新定义了计算机、数学与现代科学的可能性。他的“存储程序”理念不仅开创了计算机时代，更象征着人类用逻辑与数学征服复杂性的永恒追求。

**冯•诺依曼计算机（Von Neumann Architecture）**

冯·诺依曼计算机是现代计算机的理论基础，由美籍匈牙利科学家**约翰•冯•诺依曼**（John von Neumann）在1945年提出。其核心思想是“**存储程序**”，即程序和数据共同存储在计算机的同一存储器中，通过逐条读取指令并执行来完成计算任务。这一架构至今仍是绝大多数通用计算机的设计蓝本。

**核心思想与历史背景**

1.**核心思想**：

1. **存储程序**：程序指令和数据以二进制形式存储在同一个存储器中。
2. **顺序执行**：CPU按顺序从内存中读取指令并逐条执行。
3. **五大部件**：计算机由运算器、控制器、存储器、输入设备和输出设备组成。

2.**历史背景**：

1. 在冯·诺依曼之前，计算机（如ENIAC）的程序通过物理接线或开关设置，修改程序需重新布线。
2. 冯·诺依曼参与EDVAC项目时提出此架构，解决了程序动态加载的难题，极大提升了计算机的灵活性和通用性。

**五大核心部件**

1.**运算器（Arithmetic Logic Unit, ALU）**

1. **功能**：执行算术运算（加减乘除）和逻辑运算（与或非）。
2. **依赖**：从控制器接收指令，从存储器或寄存器获取操作数。

2.**控制器（Control Unit, CU）**

1. **功能**：从存储器读取指令，解码并生成控制信号，协调各部件工作。
2. **关键机制**：程序计数器（PC）指向下一条待执行指令的地址。

3.**存储器（Memory）**

1. **功能**：统一存储程序和数据，按地址访问。
2. **特点**：分为只读存储器（ROM）和随机存取存储器（RAM）。

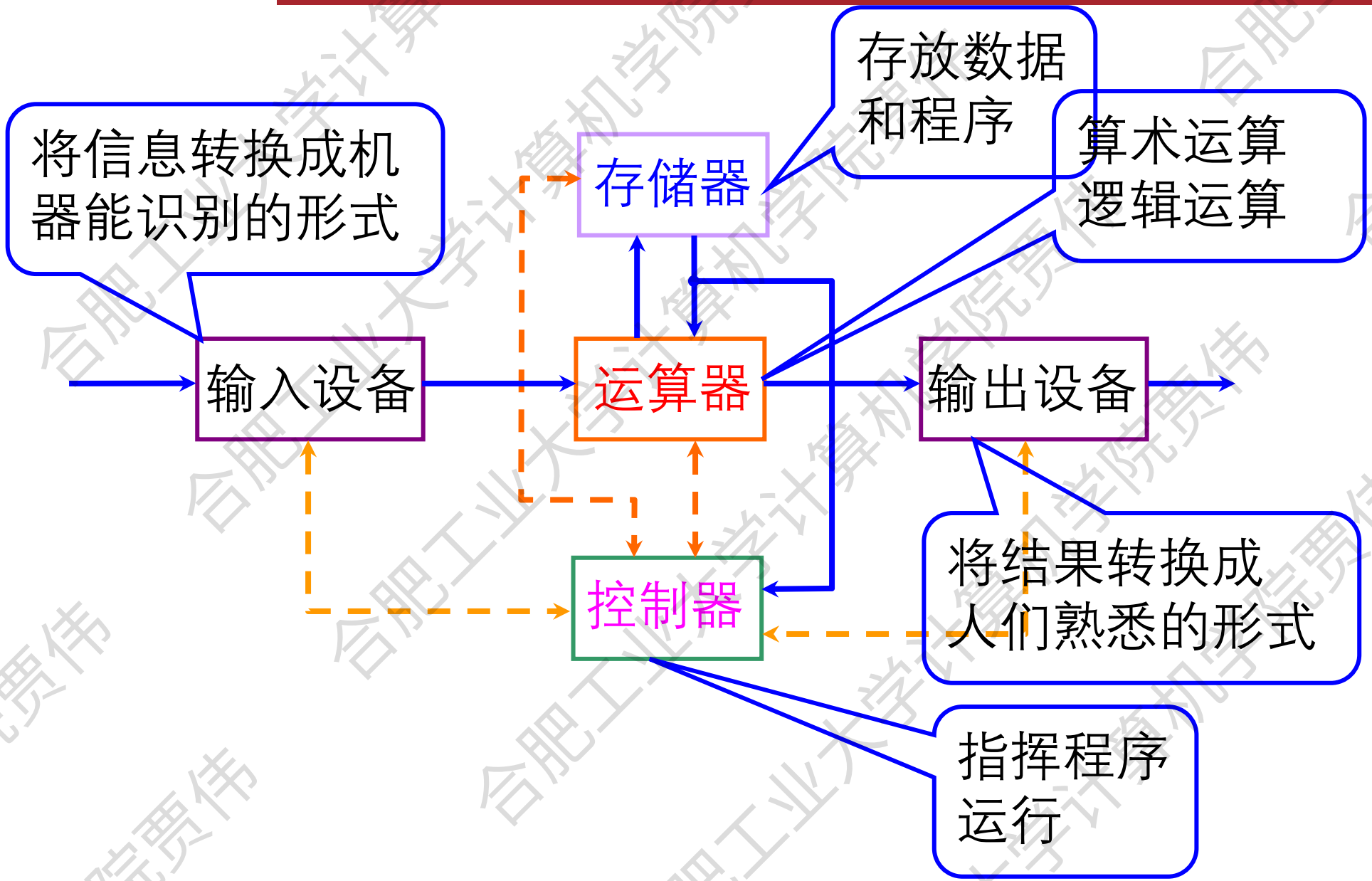
4.**输入设备（Input Devices）**

1. **功能**：将外部数据或指令转换为计算机可处理的信号（如键盘、鼠标）。

5.**输出设备（Output Devices）**

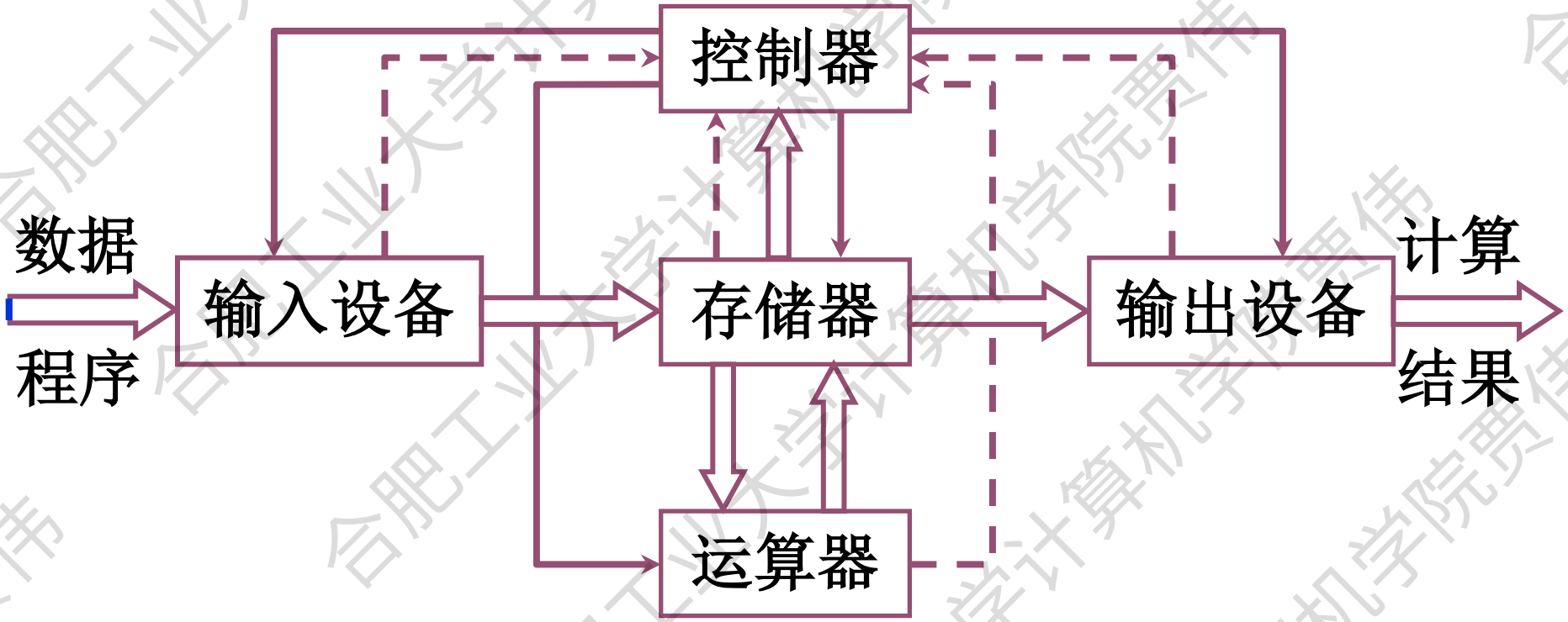
1. **功能**：将处理结果转换为人类可感知的形式（如显示器、打印机）。

合肥工业大学计算机学院贾伟



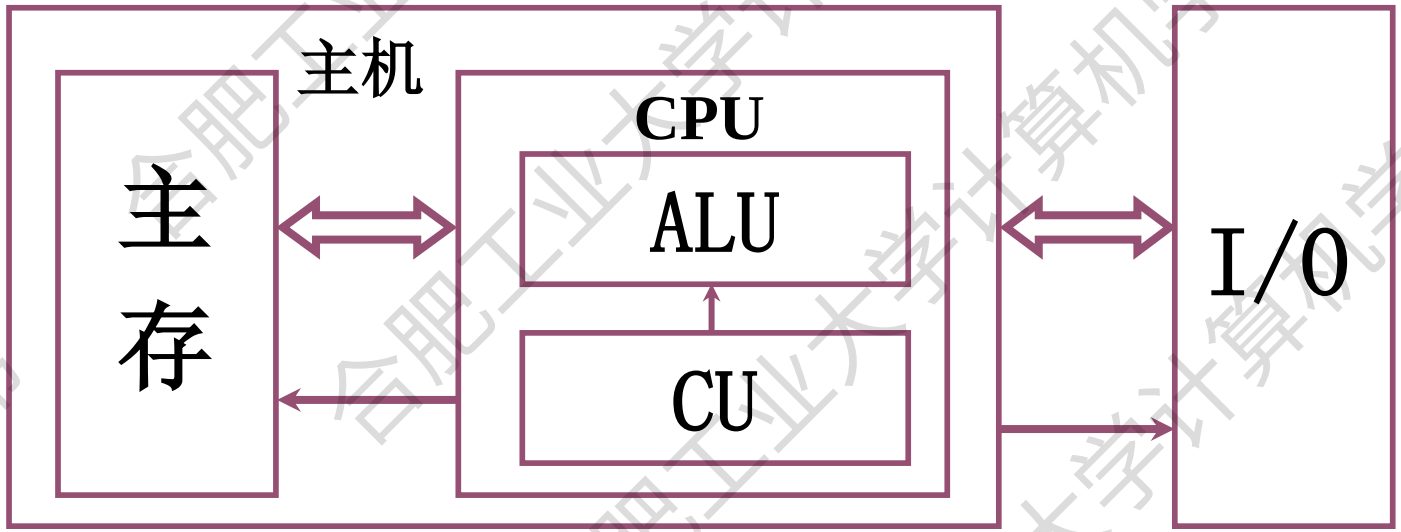

合肥工业大学计算机学院贾伟

以存储器为中心的计算机硬件框图(现代)




合肥工业大学计算机学院贾伟

运算器 ALU
控制器 CU
} CPU
存储器
{ 主存
辅存
} 主机
输入设备
输出设备
} I/O
} 硬件



**工作流程（冯•诺依曼循环）**

1.**取指令（Fetch）**

1. 控制器根据程序计数器（PC）的地址，从存储器中读取指令。

2.**解码指令（Decode）**

1. 控制器解析指令的操作码（Opcode），确定需要执行的操作（如加法、跳转）。

3.**执行指令（Execute）**

1. 运算器根据指令对数据进行处理，结果可能写回存储器或寄存器。

4.**更新程序计数器**

1. PC指向下一条指令地址（顺序执行或跳转）。

5.**重复循环**

1. 周而复始，直到程序结束或收到终止信号。

合肥工业大学计算机学院贾伟

**冯•诺依曼架构的影响与意义**

1.**通用性**

1. 程序可动态修改和加载，使计算机能够执行多种任务（从科学计算到图形渲染）。

2.**标准化设计**

1. 五大部件的分工成为现代计算机设计的模板（如x86、ARM架构均基于此）。

3.**存储程序革命**

1. 摆脱了早期计算机“专机专用”的局限性，推动了软件产业的诞生。

合肥工业大学计算机学院贾伟

**冯•诺依曼架构的局限性**

1.**冯•诺依曼瓶颈（Von Neumann Bottleneck)**

1. CPU与存储器之间的带宽限制导致性能瓶颈（CPU处理速度远高于内存访问速度）。
2. **解决方案**：引入高速缓存（Cache）、多级存储层次、并行计算。

2.**顺序执行限制**

1. 指令逐条执行，难以充分利用硬件并行性。
2. **解决方案**：流水线技术、超标量（Superscalar）架构、多核处理器。

3.**安全性问题**

1. 程序与数据共享同一存储空间，可能被恶意代码篡改（如缓冲区溢出攻击）。

合肥工业大学计算机学院贾伟

**冯•诺依曼架构的现代发展**

1.**改进存储层次**

1. 使用L1/L2/L3缓存、SSD等缩短数据访问延迟。

2.**并行计算扩展**

1. GPU（图形处理器）和TPU（张量处理器）通过大规模并行计算突破瓶颈。

3.**混合架构**

1. 结合哈佛架构（Harvard Architecture，程序与数据分离存储）的优势，如现代CPU的分离指令/数据缓存。

合肥工业大学计算机学院贾伟

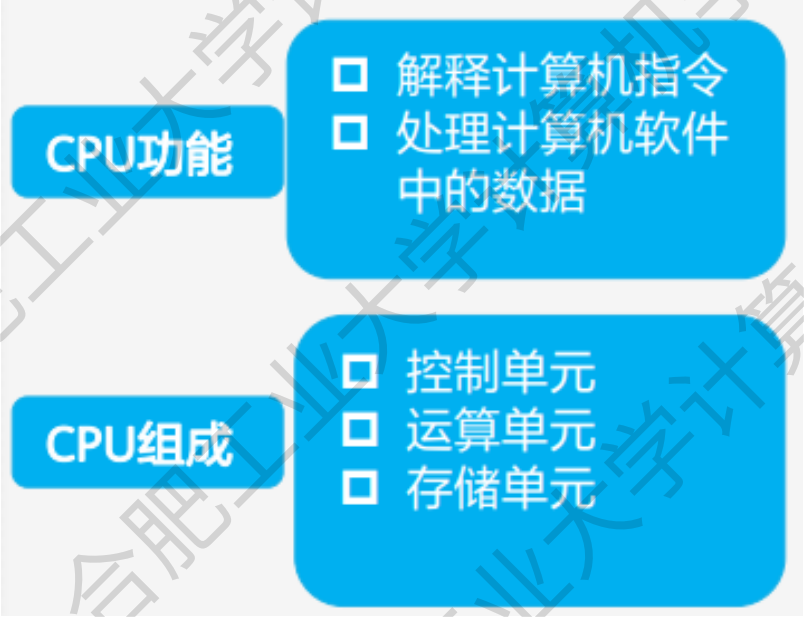
## 冯•诺依曼 vs. 哈佛架构

| 特征 | 冯•诺依曼架构 | 哈佛架构 |
| --- | --- | --- |
| **存储分离** | 程序与数据共享同一存储器 | 程序与数据分别存储在不同存储器 |
| **总线设计** | 单一总线传输指令和数据 | 独立指令总线和数据总线 |
| **应用场景** | 通用计算机（如PC、服务器） | 嵌入式系统、DSP（数字信号处理器） |
| **性能优势** | 灵活性高，易于编程 | 并行访问指令和数据，速度更快 |

冯·诺依曼架构奠定了现代计算机的基石，其“存储程序”思想彻底改变了计算方式。尽管存在瓶颈，但通过技术创新（如缓存、并行化），它仍是计算机设计的核心范式。理解这一架构，是掌握计算机组成原理和系统优化的关键起点。

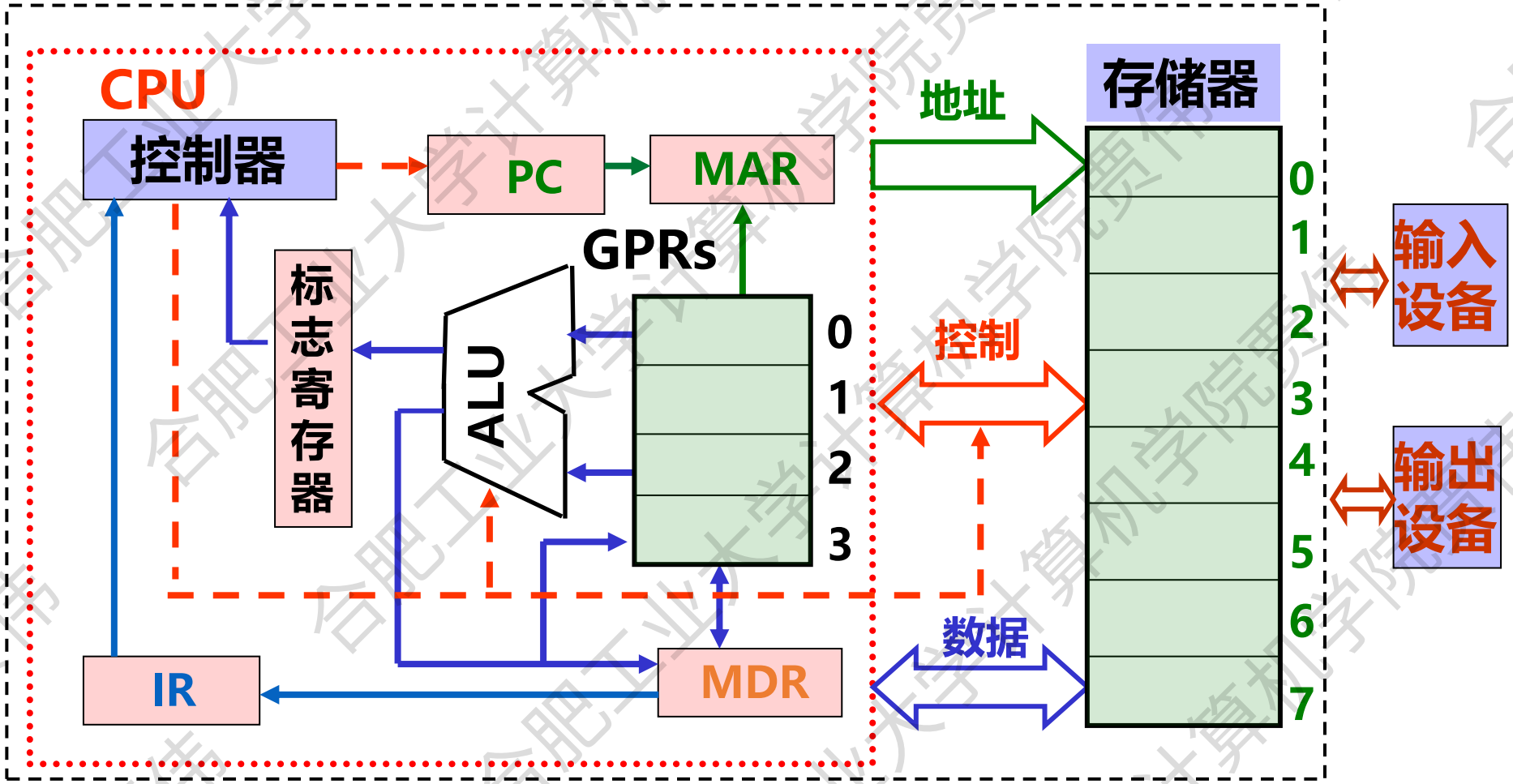
合肥工业大学计算机学院贾伟

- 中央处理器 （Central Processing Unit，CPU）是计算机的主要设备之一，功能主要是解释计算机指令 以及处理计算机软件中的数据。
- 中央处理器主要包括控制器，运算器，高速缓冲存储器（Cache）及实现它们之间联系的数据、控制及 状态的总线。
- 它与主存（Memory）和输入/输出设备合称为电子计算机三大核心部件。



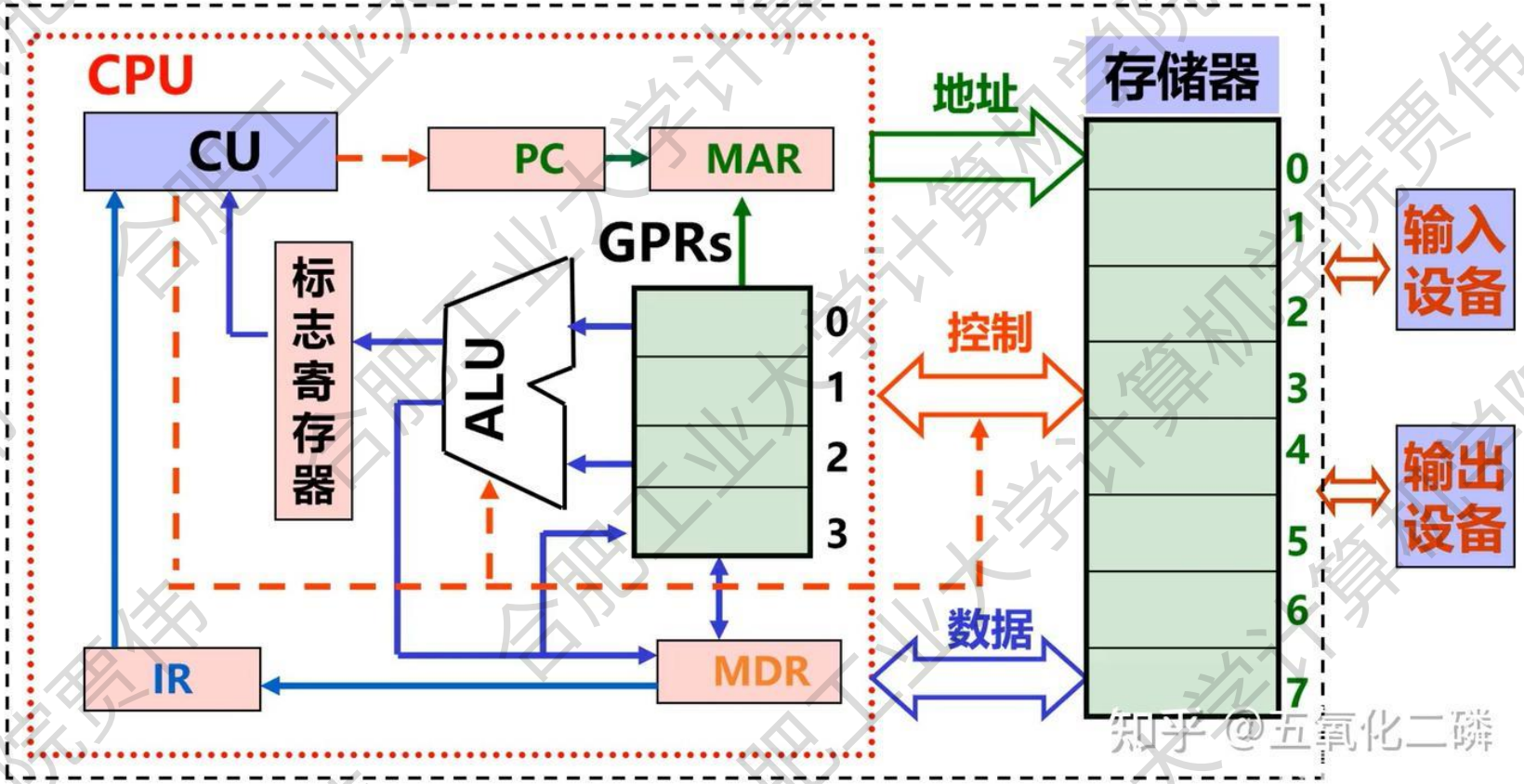

合肥工业大学计算机学院贾伟

# 现代计算机结构模型


CPU
控制器
PC
MAR
地址
存储器
GPRs
标志寄存器
ALU
0
1
2
3
控制
输入设备
输出设备
IR
MDR
数据
0
1
2
3
4
5
6
7

# 现代计算机结构模型

CPU：中央处理器；PC：程序计数器；MAR：存储器地址寄存器
ALU：算术逻辑部件；IR：指令寄存器；MDR：存储器数据寄存器
GPRs：通用寄存器组（由若干通用寄存器组成，早期就是累加器）




知乎 @五氧化二磷

**IR：**指令寄存器。用于存放当前正在执行的指令。

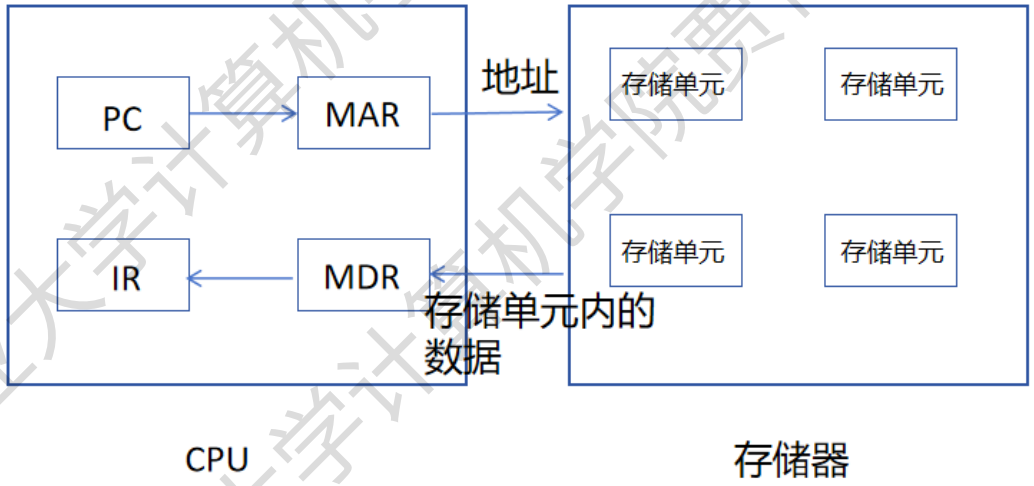
**MAR：**主存地址寄存器。存在于CPU中，存放的是将要访问主存的地址。注意，MAR之中存放的只有地址，即存储单元的索引地址。

**MDR：**主存数据寄存器。存在于CPU中，存放的是某一个存储单元。

**MAR和MDR功能作用辨析：**假设主存是一个含有若干存储单元的存储器，我们这里给存储单元进行标号，0、1、2...，此时MAR的作用就是找到这个标号所对应的单元，但是并不知道这个单元内部的东西。找到这个单元之后，存储器把这个单元里面的数据内容放到MDR中。

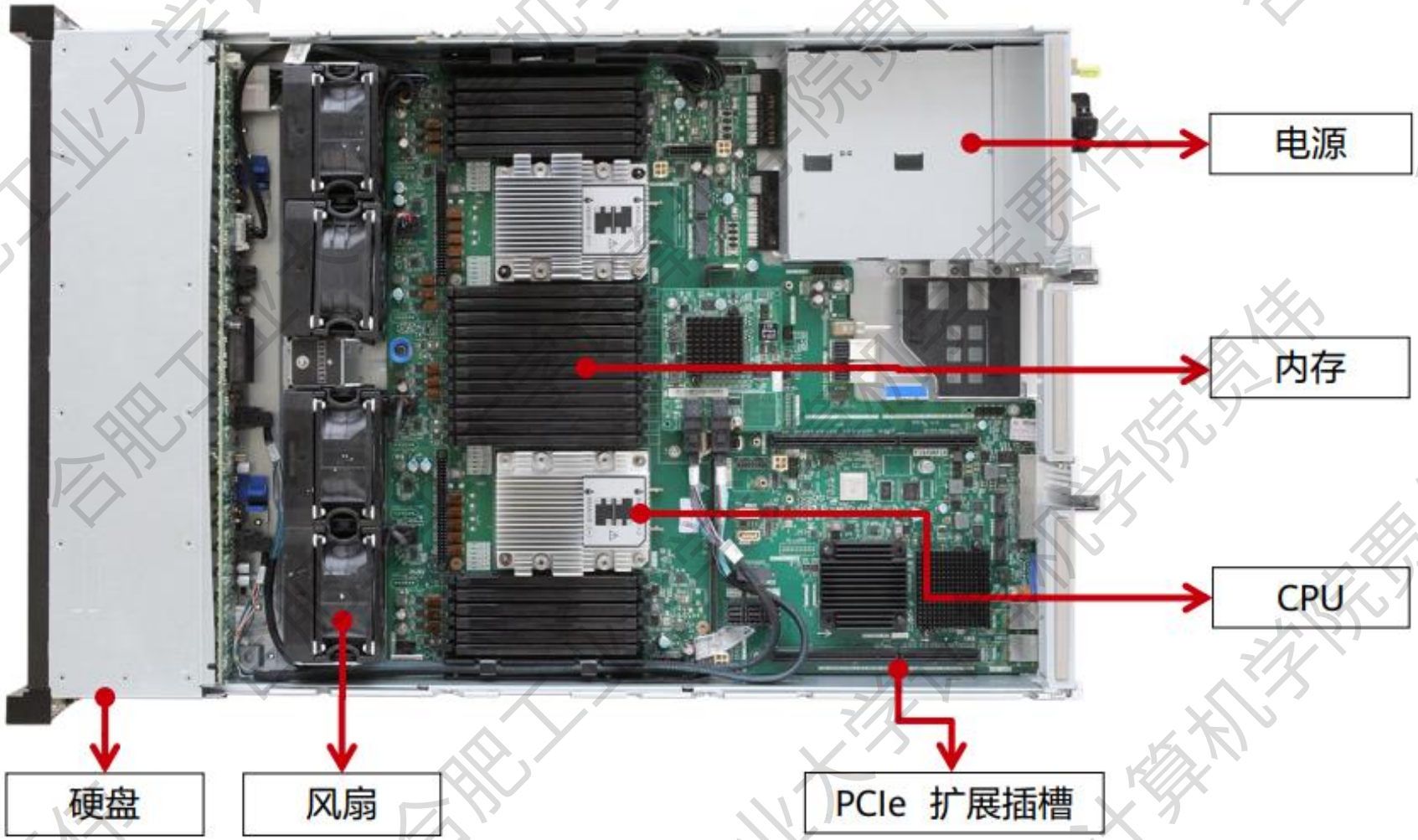
**四者之间的关系**

CPU从PC中取出指令所在的地址单元（取指阶段），之后进行译码，将该地址放进MAR中。MAR找到对应的存储单元，读取数据并将其放到MDR中，此时MDR存放的是指令加数据。之后把指令部分放到IR中进行指令操作，数据放到寄存器或者ACC中计算。



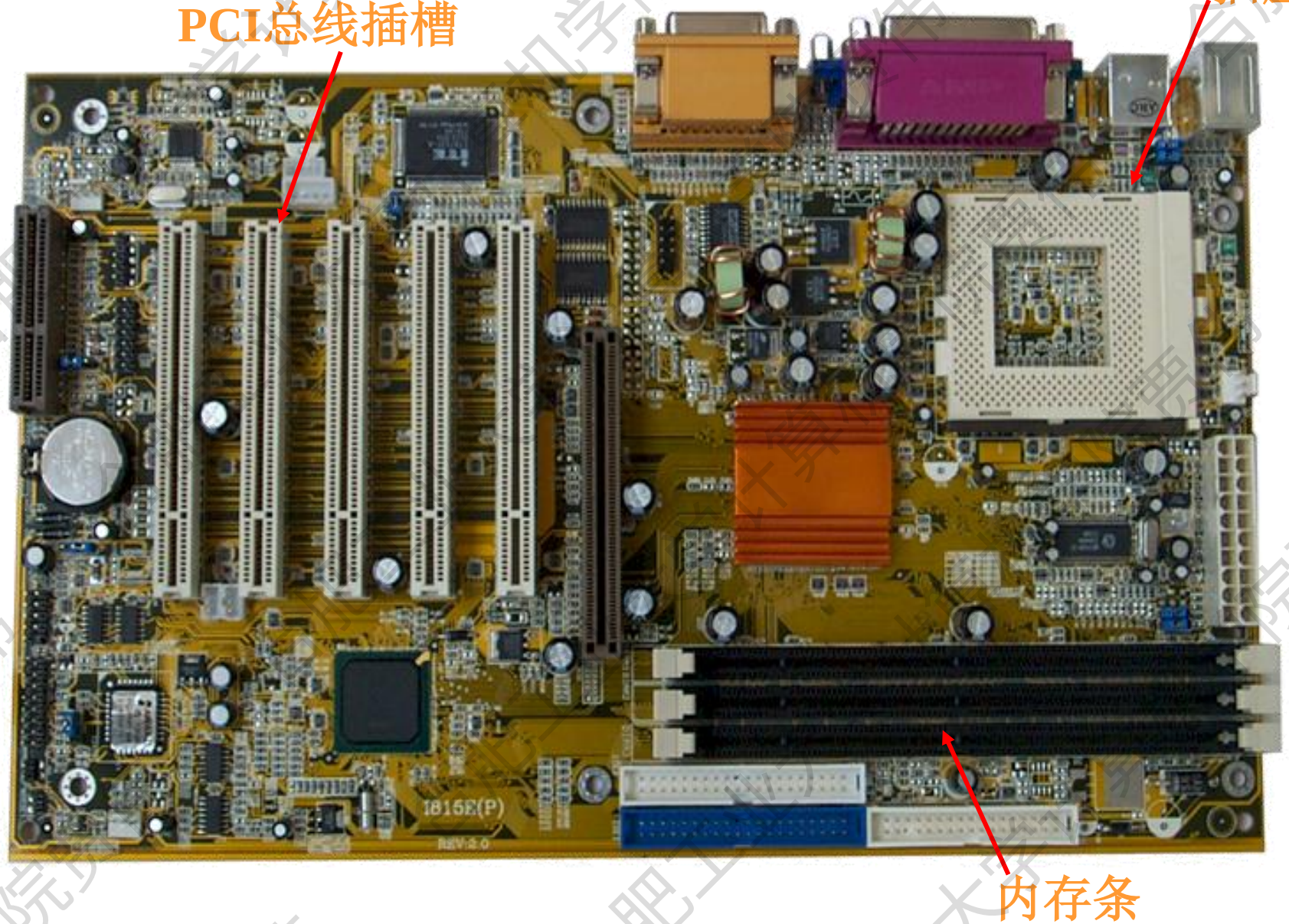



合肥工业大学计算机学院贾伟




合肥工业大学计算机学院贾伟

# PC主板




合肥工业大学计算机学院贾伟

# 计算机的工作步骤

## 1. 上机前的准备

- 建立数学模型
- 确定计算方法

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \cdots$$

$$\sqrt{x} = \frac{1}{2}\left(y_n + \frac{x}{y_n}\right)\left(n = 0, 1, 2, \cdots\right)$$

- 编制解题程序

程序 — 运算的 全部步骤

指令 — 每 一个步骤

合肥工业大学计算机学院贾伟

# 编程举例

计算 $ax^2 + bx + c = (ax + b)x + c$

取$x$ 至运算器中
乘以$x$ 在运算器中
乘以a 在运算器中
存$ax^2$ 在存储器中
取b 至运算器中
乘以$x$ 在运算器中
加$ax^2$ 在运算器中
加c 在运算器中

取$x$ 至运算器中
乘以a 在运算器中
加b 在运算器中
乘以$x$ 在运算器中
加c 在运算器中

合肥工业大学计算机学院贾伟

# 指令格式举例

| 操作码 | 地址码 |
| --- | --- |

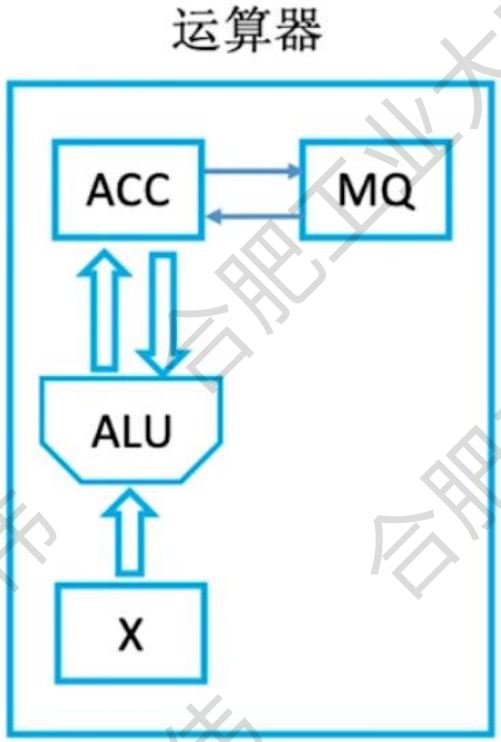
| | | |
| --- | --- | --- |
| 取数 | α | [α] → ACC |
| 000001 | 0000001000 | |
| 存数 | β | [ACC] → β |
| 加 | γ | [ACC]+[γ] → ACC |
| 乘 | δ | [ACC]×[δ] → ACC |
| 打印 | σ | [σ] → 打印机 |
| 停机 | | |

合肥工业大学计算机学院贾伟

# 计算 $ax^2 + bx + c$ 程序清单

| 指令和数据存于主存单元的地址 | 指令 | | 注释 |
|---|---|---|---|
| | 操作码 | 地址码 | |
| 0 | 000001 | 0000001000 | 取数*x*至ACC |
| 1 | 000100 | 0000001001 | 乘a得a*x*存于ACC中 |
| 2 | 000011 | 0000001010 | 加b得a*x*+b,存于ACC中 |
| 3 | 000100 | 0000001000 | 乘*x*得（a*x*+b)*x*,存于ACC中 |
| 4 | 000011 | 0000001011 | 加c得$ax^2 + bx + c$,存于ACC |
| 5 | 000010 | 0000001100 | 将$ax^2 + bx + c$存于主存单元 |
| 6 | 000101 | 0000001100 | 打印 |
| 7 | 000110 | | 停机 |
| 8 | *x* | | 原始数据*x* |
| 9 | a | | 原始数据a |
| 10 | b | | 原始数据b |
| 11 | c | | 原始数据c |
| 12 | | | 存放结果 |

# 运算器的基本组成




运算器：用于实现算术运算（如：加减乘除）、逻辑运算（如：与或非）

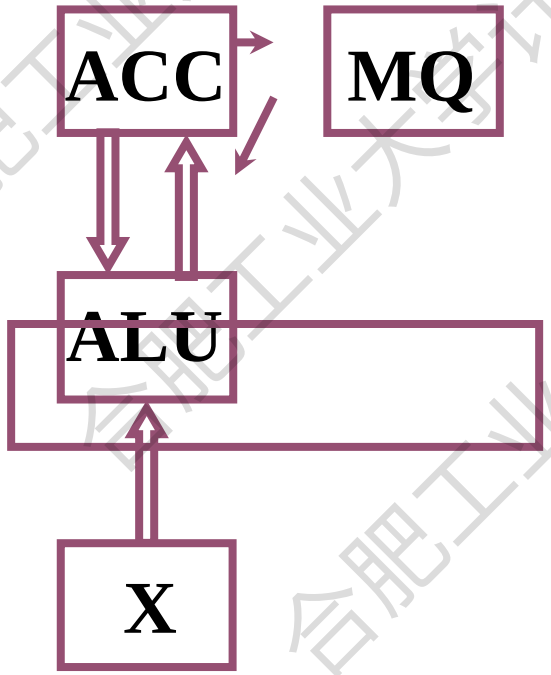
ACC： 累加器，用于存放操作数，或运算结果。
MQ： 乘商寄存器，在乘、除运算时，用于存放操作数或运算结果。
X： 通用的操作数寄存器，用于存放操作数
ALU： 算术逻辑单元，通过内部复杂的电路实现算数运算、逻辑运算

| | 加 | 减 | 乘 | 除 |
|---|---|---|---|---|
| ACC | 被加数、和 | 被减数、差 | 乘积高位 | 被除数、余数 |
| MQ | | | 乘数、乘积低位 | 商 |
| X | 加数 | 减数 | 被乘数 | 除数 |

Accumulator
Multiple-Quotient Register
Arithmetic and Logic Unit

# 2. 计算机的解题过程

## (1)运算器的基本组成及操作过程



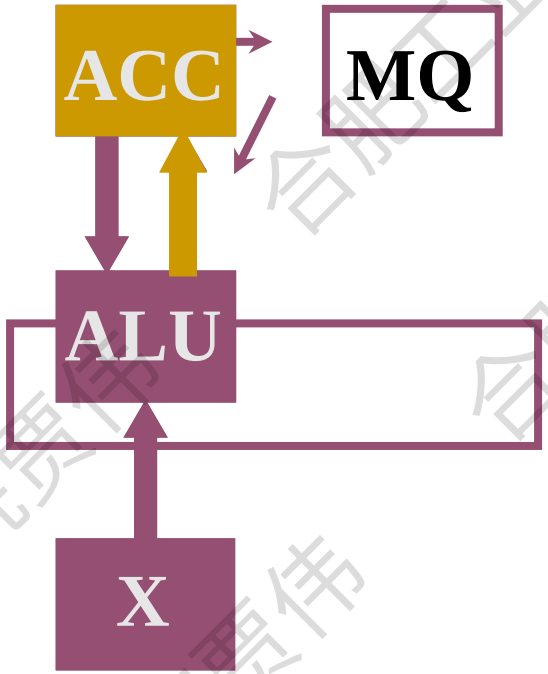

运算器

| | ACC | MQ | X |
|---|---|---|---|
| 加法 | 被加数<br>和 | | 加数 |
| 减法 | 被减数<br>差 | | 减数 |
| 乘法 | 乘积高位 | 乘数<br>乘积低位 | 被乘数 |
| 除法 | 被除数<br>余数 | 商 | 除数 |

合肥工业大学计算机学院贾伟

# (1)运算器的基本组成及操作过程

## ① 加法操作过程




运算器

指令 | 加 | M

初态 ACC 被加数

[M] ⟶ X

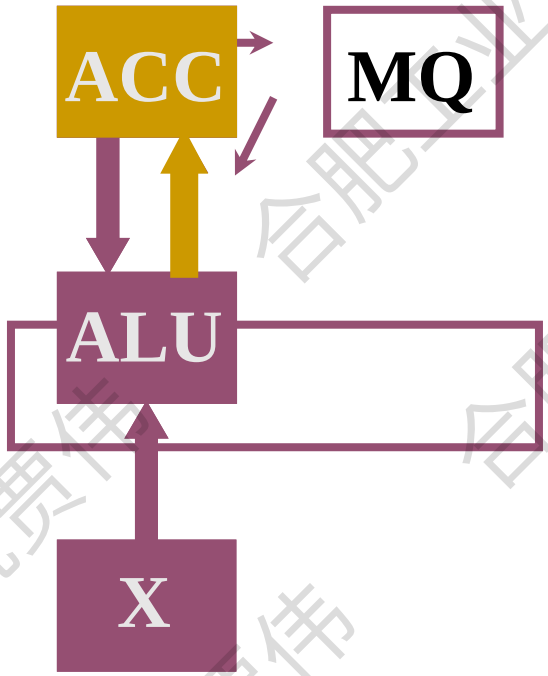
[ACC]+[X] ⟶ ACC

加法指令
初态:ACC存放被加数
【M】到X中
【ACC】+【X】->ACC

将内存M中的数字送到X中，通过ALU将X与ACC相加，得到的值送到ACC中。这些动作都是由控制器来控制的。

# (1)运算器的基本组成及操作过程

## ② 减法操作过程




运算器

指令 | 减 | M

初态 ACC 被减数

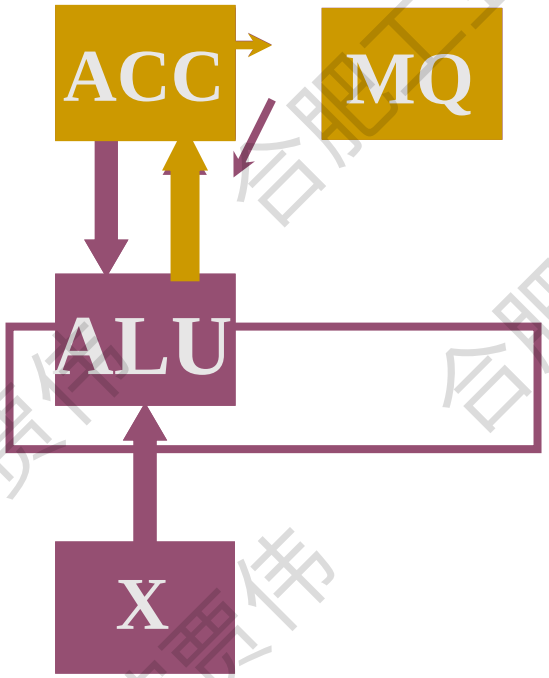
[M] ⟶ X

[ACC]-[X] ⟶ ACC

减法指令
初态:ACC存放被减数
【M】到X中
【ACC】-【X】->ACC

合肥工业大学计算机学院贾伟

# ③ 乘法操作过程




运算器

指令 | 乘 | M

初态 ACC 被乘数

[M] → MQ

[ACC] → X

0 → ACC

[X]×[MQ] → ACC//MQ

乘法指令
初态:ACC存放被乘数
【M】到MQ中
【ACC】到X中
0到ACC中
【X】*【MQ】->ACC//MQ

# ③ 乘法操作过程

①初态：被乘数放在ACC中

②假设乘法指令中，乘数的地址为M，那么[M]->MQ

③根据约定，被乘数要放到X中，所以[ACC]->X

④乘法通过累加和移位进行，所以要让ACC保存累加的内容。

在这之前，要将ACC清零，即0->ACC

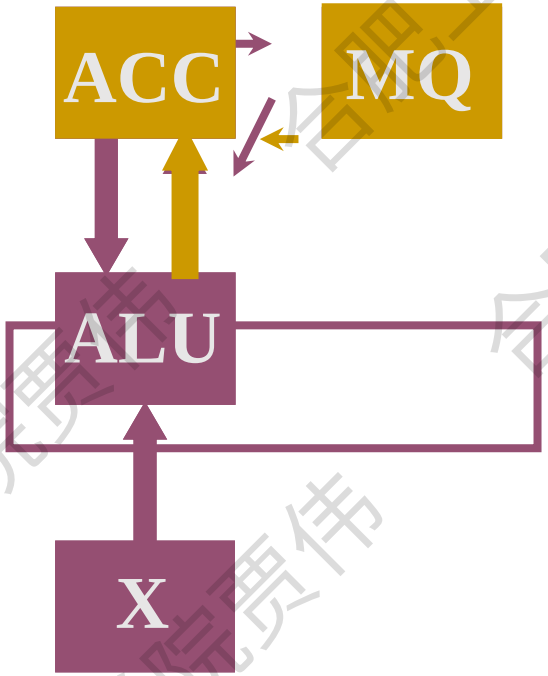
⑤[X]×[MQ]->ACC//MQ

“//”表示寄存器的连接关系

合肥工业大学计算机学院贾伟

# (1)运算器的基本组成及操作过程

## ④ 除法操作过程




运算器

指令 | 除 | M

初态 ACC 被除数

[M] ⟶ X

[ACC] ÷ [X] ⟶ MQ

余数在ACC中

除法指令
初态:ACC存放被除数
【M】到X中
【X】/【MQ】->MQ
余数在ACC中

# (1)运算器的基本组成及操作过程

**除法**

①ACC中保存被除数
②假设除法指令中，除数保存在地址M中，那么[M]->X
③ALU执行除法运算，商存在MQ中，余数放在ACC中

可以看到乘法和除法中用到了MQ，乘法用MQ保存乘数，除法用MQ保存商，所以叫做乘商寄存器

合肥工业大学计算机学院贾伟

# (2)存储器的基本组成




存储体 – 存储单元 – 存储元件 （0/1）

[教学楼 – 教室 – 座位（无人/有人）]

存储单元 存放一串二进制代码

存储字 存储单元中二进制代码的组合

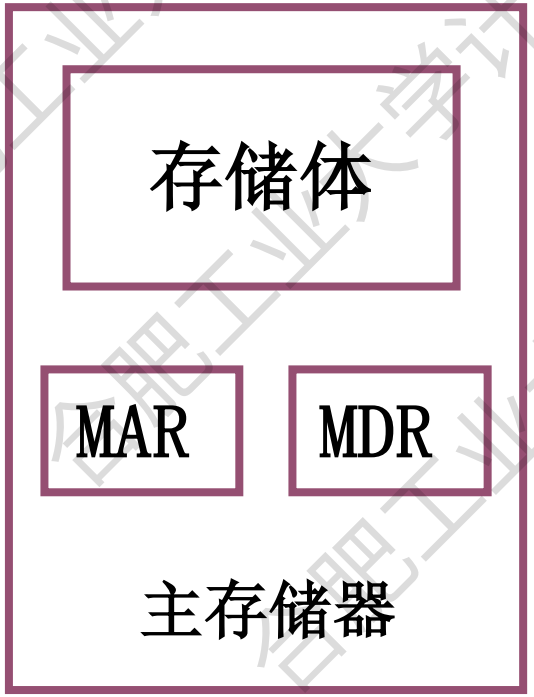
存储字长 存储单元中二进制代码的位数

每个存储单元赋予一个地址号

按地址寻访

合肥工业大学计算机学院贾伟

# (2)存储器的基本组成



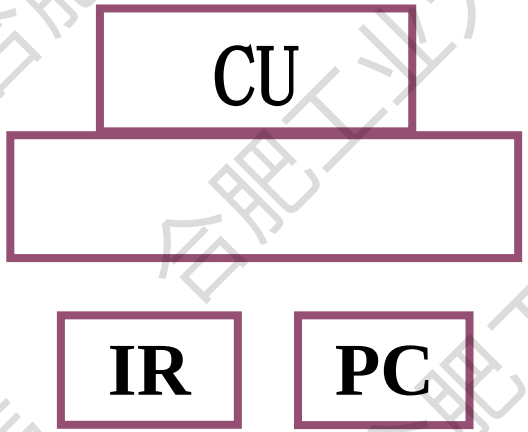

MAR　存储器地址寄存器
反映存储单元的个数

MDR　存储器数据寄存器
反映存储字长

设 MAR = 4 位
MDR = 8 位
存储单元个数 16
存储字长 8

合肥工业大学计算机学院贾伟

# (3)控制器的基本组成




解释一条指令
- 取指令　PC
- 分析指令　IR
- 执行指令　CU

取指　访存

执行　访存

PC 存放当前欲执行指令的地址

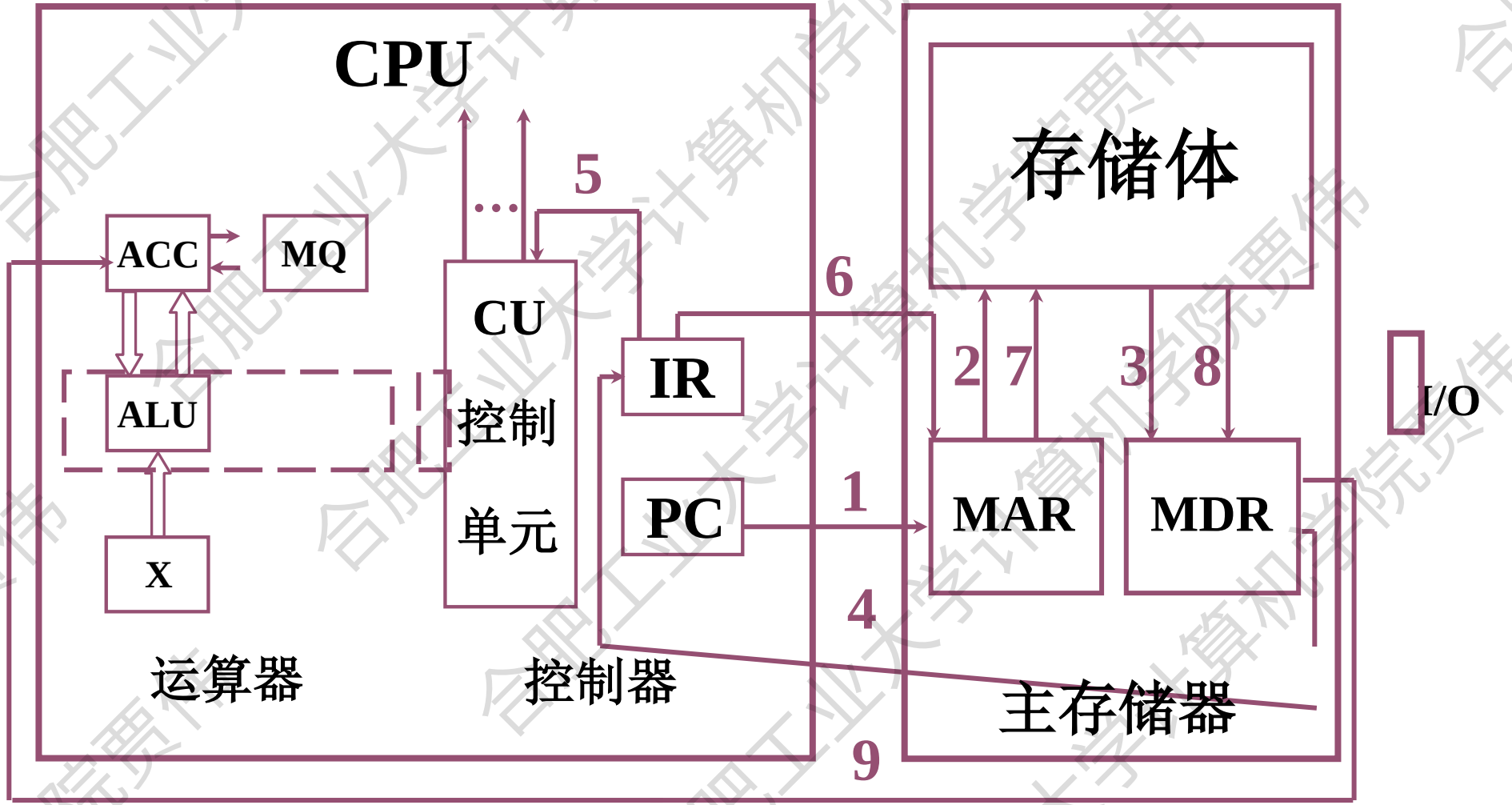
具有计数功能（PC）+1→PC

IR 存放当前欲执行的指令

合肥工业大学计算机学院贾伟

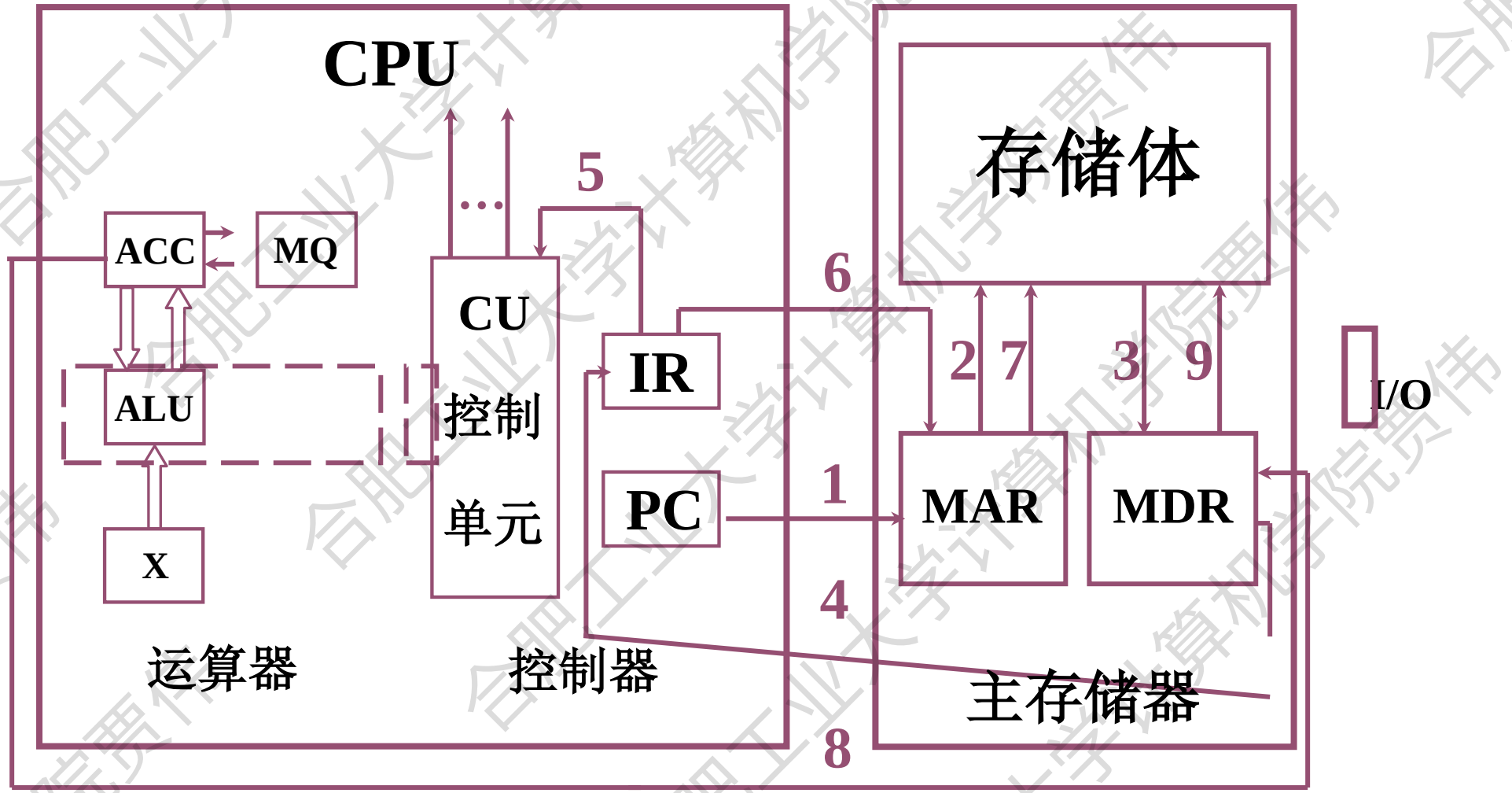
# (4) 主机完成一条指令的过程

以取数指令为例



# (4) 主机完成一条指令的过程

以存数指令为例



# (5) $ax^2 + bx + c$ 程序的运行过程

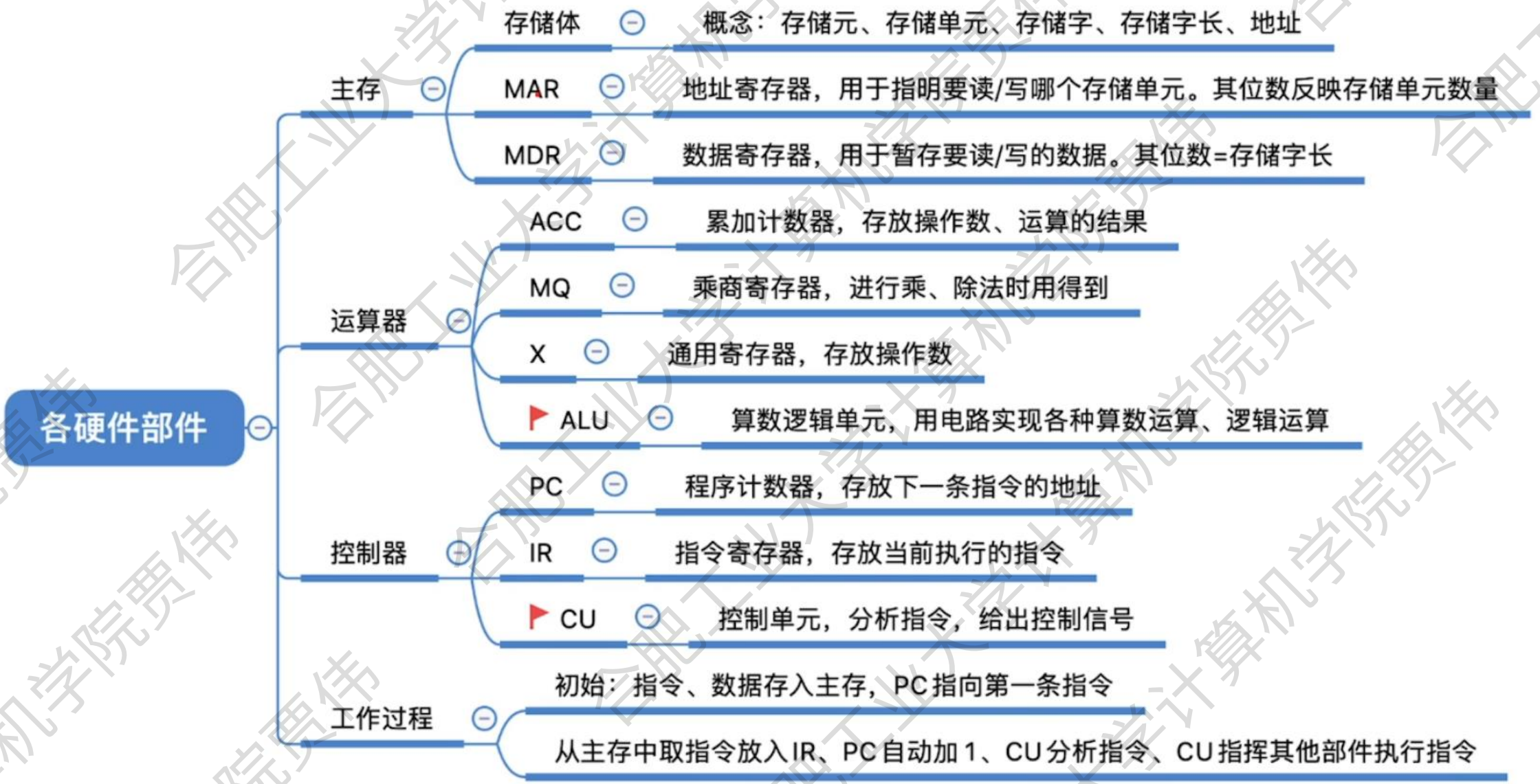
- 将程序通过输入设备送至计算机
- 程序首地址 → PC
- 启动程序运行
- 取指令 PC → MAR → M → MDR → IR ，(PC)+1 → PC
- 分析指令 OP(IR) → CU
- 执行指令 Ad(IR) → MAR → M → MDR → ACC

  ⋮

- 打印结果
- 停机

合肥工业大学计算机学院贾伟

# 知识回顾与重要考点


各硬件部件
主存
存储体
概念：存储元、存储单元、存储字、存储字长、地址
MAR
地址寄存器，用于指明要读/写哪个存储单元。其位数反映存储单元数量
MDR
数据寄存器，用于暂存要读/写的数据。其位数=存储字长
运算器
ACC
累加计数器，存放操作数、运算的结果
MQ
乘商寄存器，进行乘、除法时用得到
X
通用寄存器，存放操作数
ALU
算数逻辑单元，用电路实现各种算数运算、逻辑运算
控制器
PC
程序计数器，存放下一条指令的地址
IR
指令寄存器，存放当前执行的指令
CU
控制单元，分析指令，给出控制信号
工作过程
初始：指令、数据存入主存，PC指向第一条指令
从主存中取指令放入IR、PC自动加1、CU分析指令、CU指挥其他部件执行指令

# 计算机系统性能评价

- 制造成本（manufacturing cost）
- 衡量计算机性能的基本指标
  - 响应时间（response time）：从作业提交开始到作业完成所花的时间
    - 执行时间（execution Time）、等待时间（latency）
    - 一个程序的响应时间除了程序包含的指令在CPU上执行所花的时间，还包括磁盘访问时间、存储器访问时间、输入输出操作所需时间以及操作系统运行这个程序所花的额外开销
  - 吞吐量（throughput）：单位时间内所完成的工作量
    - 带宽（bandwidth）
- 计算机性能测量
- 指令执行速度（MIPS、MFLOPS）
- 基准程序（Benchmark）

合肥工业大学计算机学院贾伟

# 计算机性能的基本评价指标

- 计算机有两种不同的性能
  - Time to do the task
    - 响应时间（response time）
    - 执行时间（execution time）
    - 等待时间或时延（latency）
  - Tasks per day, hour, sec, ns. ..
    - 吞吐率（throughput）
    - 带宽（bandwidth）
- 基本的性能评价标准是：CPU的执行时间

不同应用场合用户关心的性能不同：

要求吞吐率高的场合，例如：

多媒体应用（音/视频播放要流畅）

要求响应时间短的场合：例如：

事务处理系统（存/取款速度要快）

要求吞吐率高且响应时间短的场合：

ATM、文件服务器、Web服务器等

" X is n times faster than Y" means

$$\frac{\text{ExTime(Y)}}{\text{ExTime(X)}} = \frac{\text{Performance(X)}}{\text{Performance(Y)}}$$

相对性能用执行时间的倒数来表示！

合肥工业大学计算机学院贾伟

# 计算机性能的测量

- 比较计算机的性能时，用执行时间来衡量
  - 完成同样工作量所需时间最短的那台计算机就是性能最好的
  - 处理器时间往往被多个程序共享使用，因此，用户感觉到的程序执行时间并不是程序真正的执行时间
  - 通常把用户感觉到的响应时间分成两个时间：
    - CPU时间：指CPU真正花在程序执行上的时间。又包括两部分：
      - **用户CPU时间：用来运行用户代码的时间**
      - **系统CPU时间：为了执行用户程序而需要运行操作系统程序的时间**
    - 其他时间：指等待I/O操作完成或CPU花在其他用户程序的时间
  - 系统性能和CPU性能不等价，有一定的区别
    - 系统性能(System performance)：系统响应时间，与CPU外的其他部分也都有关系
    - CPU性能(CPU performance)：用户CPU时间
  - 本章主要讨论CPU性能，即：CPU真正用在用户程序执行上的时间

合肥工业大学计算机学院贾伟

# 计算机系统性能评价

## 计算机硬件的三个主要性能指标：

1、机器字长：CPU 一次能处理数据的位数，通常与 CPU 中 寄存器的位数 有关。
（64位机器）机器字长会影响机器的运算速度。

2、运算速度：主频：机器内部主时钟的频率，衡量机器速度的重要参数。
（即计算机的心跳次数）它的倒数是CPU时钟周期。

MIPS：每秒执行百万条指令
MFLOPS：每秒执行百万次浮点运算
CPI：执行一条指令所需时钟周期数

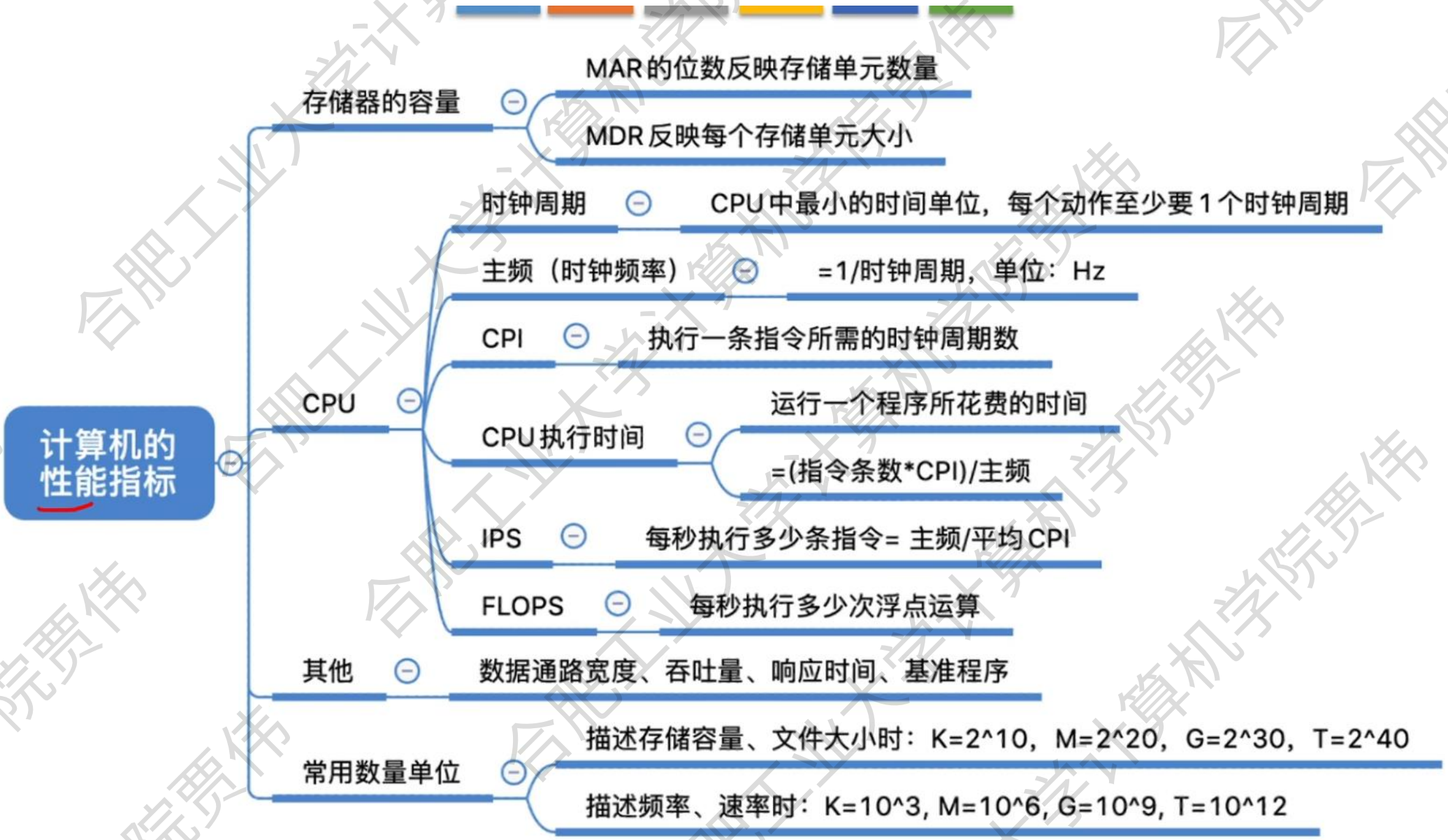
3、存储容量：存放二进制数据的总位数，用字节表示最小存储单位。
存储单元个数 × 一个存储单元位数　　例如：64 K × 32位

## 其它性能指标：

数据通路宽度：数据总线一次并行传送的位数，它关系到数据的传送能力。
吞吐量：指数据流入、处理和流出系统的速率。主要取决于主存的存取周期。
响应时间：计算机系统对特定事件的响应时间。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价


计算机的性能指标
存储器的容量
MAR的位数反映存储单元数量
MDR反映每个存储单元大小
CPU
时钟周期
CPU中最小的时间单位，每个动作至少要1个时钟周期
主频（时钟频率）
=1/时钟周期，单位：Hz
CPI
执行一条指令所需的时钟周期数
CPU执行时间
运行一个程序所花费的时间
=(指令条数*CPI)/主频
IPS
每秒执行多少条指令= 主频/平均CPI
FLOPS
每秒执行多少次浮点运算
其他
数据通路宽度、吞吐量、响应时间、基准程序
常用数量单位
描述存储容量、文件大小时：K=2^10，M=2^20，G=2^30，T=2^40
描述频率、速率时：K=10^3, M=10^6, G=10^9, T=10^12

# 计算机系统性能评价

**计算机中的机器字长（Machine Word Length）**

**机器字长**是计算机体系结构中的核心概念，指计算机**一次能处理的二进制数据的位数**，通常由CPU的设计决定。它是衡量计算机处理能力、数据精度和内存管理能力的关键指标。

**机器字长**是指CPU的**寄存器宽度**，即CPU一次能直接处理的二进制数据的位数。它决定了计算机的**字长架构**（如32位、64位）。

•**典型值**：8位（早期嵌入式）、16位（MS-DOS）、32位（Windows XP）、64位（现代主流）。

•我们所说的“某16位或32位机器”。其中16，32指的是机器字长，简称字长。字长一般等于通路寄存器的位数或者ALU的宽度，字长越长，数的表示范围越大，计算精度越高，计算机字长通常选定为字节（8位）的整数倍

**定义与核心特点**

1.**定义**

1. **硬件层面**：CPU内部数据通路（如ALU、寄存器）的二进制位数，即CPU单次运算可处理的比特数。
2. **软件层面**：操作系统和程序默认处理数据的单位长度（如32位或64位系统）。

2.**典型字长**

1. **4位**：早期微处理器（如Intel 4004）。
2. **8位**：经典单片机（如Intel 8080）。
3. **16位**：早期个人计算机（如Intel 8086）。
4. **32位**：现代通用计算机的基础（如x86架构）。
5. **64位**：主流高性能系统（如x86-64、ARMv8）。

# 计算机系统性能评价

## 机器字长的影响

**1. 数据处理能力**

•**运算精度**：字长越长，单次可处理的整数范围越大。

- 32位：最大无符号整数为 232−1=4,294,967,295。
- 64位：最大无符号整数为 264−1≈1.8×1019。

•**浮点运算**：字长决定浮点数的表示范围和精度（如64位双精度浮点数）。

**2. 内存寻址能力**

•**直接寻址空间**：字长决定CPU可访问的内存地址范围。

- 32位：最大寻址空间为 232字节=4GB。
- 64位：理论寻址空间为 264字节≈16EB（艾字节，1 EB = 1018 字节）。

**3. 寄存器与总线宽度**

•**寄存器大小**：通用寄存器（如x86的EAX/RAX）位数与字长一致。

•**总线宽度**：数据总线和地址总线通常与字长匹配（如64位CPU的数据总线为64位）。

**4. 软件兼容性**

•**操作系统**：64位系统可运行32位程序（通过兼容模式），但反之不行。

•**程序优化**：针对特定字长编译的软件需重新适配（如64位程序无法在32位系统运行）。

## 数据通路带宽

数据通路带宽是指数据总线一次所能并行传送信息的位数，这里所说的数据通路宽度是指外部数据总线的宽度，它与CPU内部的数据总线宽度（内部寄存器的大小）有可能不同

各个子系统通过数据总线链接形成的数据传送路径称为数据通路

# 计算机系统性能评价

## 运算速度

- 吞吐量：指系统在单位时间内处理请求的数量。
- 响应时间：从用户向计算机发送一个请求，到系统对该请求做出响应并获得所需结果的等待时间。
- CPU时钟周期：机器内部主时钟脉冲信号的宽度，是CPU工作的最小时间单位。
- 主频（CPU时钟频率）：机器内部主时钟的频率，即时钟周期的倒数。
- CPI：执行一条指令所需的时钟周期数。注意CPI是一个平均数。
- IPS：每秒执行多少条指令，IPS=主频/CPI。
- CPU 执行时间：指运行一个程序所花费的时间，取决于主频、CPI 和指令条数。
- MIPS（每秒执行多少百万条指令）：用于衡量计算机的指令执行能力。
- MFLOPS（每秒执行多少百万次浮点运算）：这是标志系统性能的最有用参数之一。
- GFLOPS（每秒执行多少十亿次浮点运算）和 TFLOPS（每秒执行多少万亿次浮点运算）：用于评估更强大的计算能力。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价

## 一、处理器（CPU）相关指标

### 1.主频（Clock Speed）

1. **定义**：CPU每秒钟的时钟周期数，单位为GHz（千兆赫兹）。
2. **影响**：主频越高，单线程任务执行速度越快（但受限于功耗和发热）。
3. **注意**：主频并非唯一决定因素，需结合架构和核心数综合判断。

### 2.核心数（Cores）与线程数（Threads）

1. **核心**：物理处理单元，多核CPU可并行处理任务。
2. **线程**：通过超线程技术（Hyper-Threading）模拟的虚拟核心，提升多任务效率。
3. **适用场景**：视频渲染、科学计算等多线程优化任务受益于多核。

### 3.IPC（Instructions Per Cycle）

1. **定义**：每个时钟周期执行的指令数，反映CPU架构效率。
2. **对比**：相同主频下，IPC越高性能越强（如ARM架构优化低功耗高IPC）。

### 4.缓存（Cache）

1. **容量与层级**：L1/L2/L3缓存容量越大、层级越合理，数据存取延迟越低。
2. **作用**：减少CPU访问内存的等待时间，显著影响密集型计算性能。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价

**时钟周期与主频的概念**

在计算机中，**时钟周期**和**主频**是描述CPU工作节奏的核心概念，它们直接关联到计算机的执行速度和性能。以下是详细的解释：

**1. 时钟周期（Clock Cycle）**

**•定义**：

时钟周期是CPU内部时钟信号完成一次完整波动（从高电平到低电平再回到高电平）所需的时间。它是CPU执行操作的最小时间单位。

- **单位**：通常用**纳秒（ns）**或**皮秒（ps）**表示。
- **符号**：$T$（周期时间）。

**•作用**：

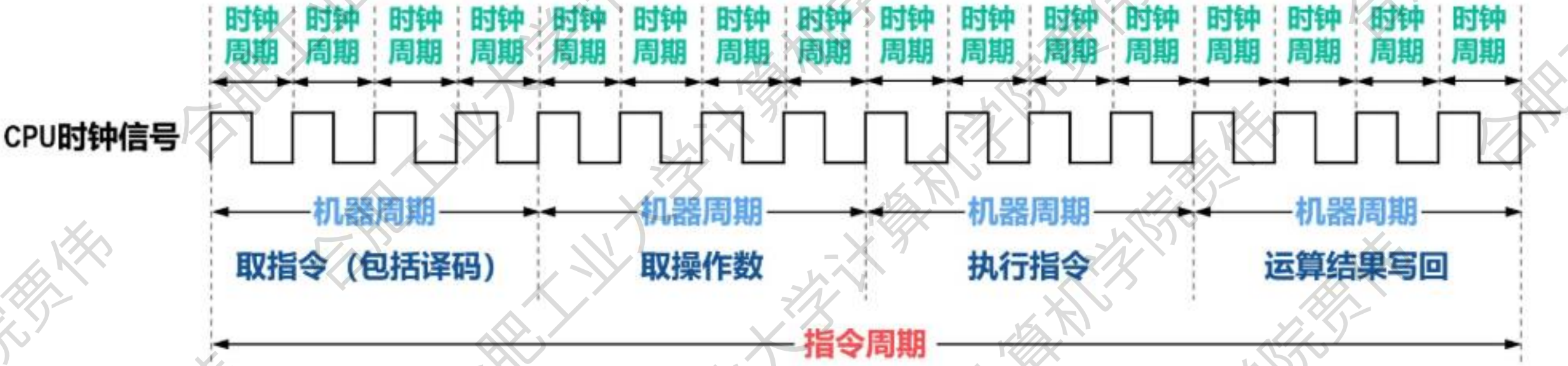
- CPU的所有操作（如取指令、解码、执行）都按时钟周期的节拍逐步推进。
- 例如：一条指令可能需要多个时钟周期完成（如简单指令需1周期，复杂指令需几十周期）。

**•示例**：

- 若主频为3 GHz，则时钟周期 $T \approx 0.333$纳秒。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价




合肥工业大学计算机学院贾伟

# CPU的性能指标




单位：微秒、纳秒...

CPU时钟周期

单位：赫兹，Hz

$$\text{CPU主频（时钟频率）} = \frac{1}{CPU\text{时钟周期}}$$

不同的指令，CPI不同。甚至相同的指令，CPI也可能有变化

CPI（Clock cycle Per Instruction）：执行一条指令所需的时钟周期数

执行一条指令的耗时 = CPI × CPU时钟周期

CPU执行时间=CPU时钟周期数/主频=（指令条数*CPI）/主频

KIPS
MIPS

IPS（ Instructions Per Second ）：每秒执行多少条指令

$$IPS = \frac{\text{主频}}{\text{平均CPI}}$$

KFLOPS
MFLOPS
GFLOPS
TFLOPS

FLOPS（Floating-point Operations Per Second）：每秒执行多少次浮点运算

注：此处K、M、G、T为数量单位

K=Kilo=千=$10^3$，M=Million=百万=$10^6$，G=Giga=十亿=$10^9$，T=Tera=万亿=$10^{12}$

# 计算机系统性能评价

## 1. CPU时钟周期的定义与基本概念

- 定义：CPU时钟周期，又称为T周期，是CPU处理操作的最基本单位。它代表CPU执行一个基本操作所需的时间段，是主频的倒数。
- 基本概念：在一个时钟周期内，CPU仅能完成一个最基本的动作，如取指令、译码、执行等。时钟周期的长短决定了CPU执行指令的速度。

## 2. CPU时钟周期与主频的关系

- 主频定义：主频是CPU的时钟频率，表示CPU内部时钟的振荡频率，通常以赫兹(Hz)为单位。例如，1GHz的主频表示CPU每秒能进行10^9次振荡。
- 关系阐述：CPU时钟周期与主频成反比关系。主频越高，意味着每秒内的振荡次数越多，因此每个时钟周期的时间就越短，CPU执行指令的速度就越快。反之，主频越低，时钟周期就越长，执行速度就越慢。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价

## 3. CPU时钟周期对系统性能的影响

- 执行效率：时钟周期的长短直接影响CPU执行指令的效率。较短的时钟周期意味着CPU能在更短的时间内完成更多的操作，从而提高系统的整体性能。
- 响应速度：在交互式应用中，较短的时钟周期有助于缩短系统响应时间，提升用户体验。
- 能耗与散热：虽然较短的时钟周期能提高性能，但也会增加CPU的能耗和散热压力。因此，在设计系统时需要综合考虑性能、能耗和散热等因素。

## 4. 优化CPU时钟周期的策略

- 提升主频：通过提高CPU的主频来缩短时钟周期，从而提高执行速度。但这种方法可能会受到技术瓶颈和能耗散热等问题的限制。
- 采用先进的微架构：现代CPU通过采用更先进的微架构来优化内部流水线和指令执行效率，从而在保持较低主频的同时获得较高的性能。
- 利用并行处理技术：通过多线程、多核等并行处理技术来同时执行多个任务或指令，从而在不增加单个时钟周期负担的情况下提高整体性能。
- 软件优化：通过编译器优化、算法改进等手段减少程序中的冗余操作和不必要的计算量，从而降低每条指令所需的时钟周期数并提高执行效率。

# 计算机系统性能评价

## 二、内存（RAM）相关指标

1.**容量（Capacity）**

1. **定义**：内存可临时存储的数据量，单位GB。
2. **影响**：大内存支持更多程序同时运行，避免频繁读写硬盘（如16GB vs. 32GB）。

2.**频率（Frequency）**

1. **定义**：内存数据传输速率，如DDR4-3200（3200MHz）。
2. **作用**：高频内存提升CPU与内存间的数据吞吐量。

3.**延迟（Latency）**

1. **参数**：CL值（CAS Latency），如CL16表示读取数据需16个时钟周期。
2. **优化**：低延迟内存对游戏和实时计算更友好。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价

## 三、存储设备（Storage）相关指标

### 1.读写速度

1. **顺序读写**：影响大文件传输速度（如视频剪辑）。
2. **随机读写**：决定小文件处理能力（如操作系统启动、数据库查询）。
3. **典型值**：
    1. HDD：约100-200 MB/s（顺序读）。
    2. SATA SSD：约500 MB/s。
    3. NVMe SSD：可达7000 MB/s（PCIe 4.0 x4）。

### 2.存储类型

1. **HDD**：机械硬盘，容量大但速度慢。
2. **SSD**：固态硬盘，速度快但价格高，寿命受写入次数限制。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价

## 四、图形处理（GPU）相关指标

### 1.浮点运算能力（TFLOPS）

1. **定义**：每秒浮点运算次数，衡量GPU计算性能（如NVIDIA RTX 4090约100 TFLOPS）。
2. **应用**：深度学习、3D渲染、科学模拟。

### 2.显存（VRAM）容量与带宽

1. **容量**：决定可处理的数据规模（如8GB显存对4K游戏更友好）。
2. **带宽**：显存与GPU核心间的数据传输速率，单位GB/s（如GDDR6X显存带宽可达1 TB/s）。

### 3.架构特性

1. **CUDA核心（NVIDIA）/ Stream处理器（AMD）**：并行计算单元数量。
2. **光线追踪核心（RT Core）**：加速光线追踪效果（如游戏和影视特效）。

合肥工业大学计算机学院贾伟

# 计算机系统性能评价

## 五、系统综合指标

1.**总线带宽（Bus Bandwidth）**

1. **定义**：CPU与内存、外设间的数据传输速率（如PCIe 5.0 x16带宽约63 GB/s）。
2. **影响**：高带宽支持更快扩展设备（如显卡、高速SSD）。

2.**能耗与散热**

1. **TDP（Thermal Design Power）**：芯片设计功耗，反映散热需求（如65W vs. 250W CPU）。
2. **适用场景**：笔记本关注低TDP以延长续航，服务器追求性能/功耗比。

3.**响应时间与吞吐量**

1. **响应时间**：单个任务从请求到完成的时间（如程序启动速度）。
2. **吞吐量**：单位时间内处理的任务量（如服务器并发处理请求数）。

合肥工业大学计算机学院贾伟

# Marketing Metrics （产品宣称指标）

**MIPS** = Instruction Count / Time x$10^6$

= Clock Rate / CPI x $10^6$

**M**illion **I**nstructions **P**er **S**econd

平均每秒执行百万条指令数

因为每条指令执行时间不同，所以MIPS总是一个平均值。

- 不同机器的指令集不同
- 程序由不同的指令混合而成
- 指令使用的频度动态变化
- **Peak MIPS:** （不实用）

用MIPS数表示性能有没有局限?

所以MIPS数不能说明性能的好坏（用下页中的例子来说明）

**MFLOPS** = FP Operations / Time x$10^6$

**M**illion **Flo**ating-point Operations **P**er **S**econd

- 与机器相关性大
- 并不是程序中花时间的部分

用MFLOPS数表示性能也有局限！

合肥工业大学计算机学院贾伟

# 浮点运算速度

- FLOPS:
  - 1 Megaflop/s指每秒执行100万次浮点运算，即10的6次方FLOPS；
  - 1 Gigaflop/s指每秒执行10亿次浮点运算，即10的9次方FLOPS；
  - 1 Teraflop/s指每秒执行1万亿次浮点运算，即10的12次方FLOPS
  - 1 Petaflop/s指每秒执行1000万亿次浮点运算，即10的15次方FLOPS
  - 1 Exaflop/s即每秒执行100万万亿次浮点运算，10的18次方FLOPS。

合肥工业大学计算机学院贾伟

# 概念及技术指标

- 存储容量：　存放二进制信息的总数量

存储单元个数 × 存储字长

主存容量

如：MAR　MDR　容量

10　8　1K × 8位

16　32　64K × 32位

1K = $2^{10}$

字节数

如：$2^{13}$ = 1KB

1Byte = $2^3$

$2^{21}$ = 256KB

辅存容量　字节数　80GB

1G = $2^{30}$

# 概念及技术指标

- 系列机：具有基本相同的体系结构，使用相同的基本指令系统的多个不同型号的计算机组成的一个产品系列。
  - 系列机的出现是计算机发展过程中的重要事件，对计算机推广应用起到重要作用。

合肥工业大学计算机学院贾伟

# 概念及技术指标

- 兼容性：指计算机软件或硬件的通用性，即使用或运行在某个型号的计算机系统中的硬件、软件也能应用于另外一个型号的计算机系统时，则称这两台计算机在硬件或软件上存在兼容性。
  - 通常，在同一个系列的不同型号的计算机之间存在兼容性
  - 当某个厂家的成功计算机系统或某些部件被其它厂家在保持相同的功能特性的前提下，通过合法渠道重新设计、组织生产并投入市场营销，这些产品称为兼容计算机或兼容部件产品。

合肥工业大学计算机学院贾伟

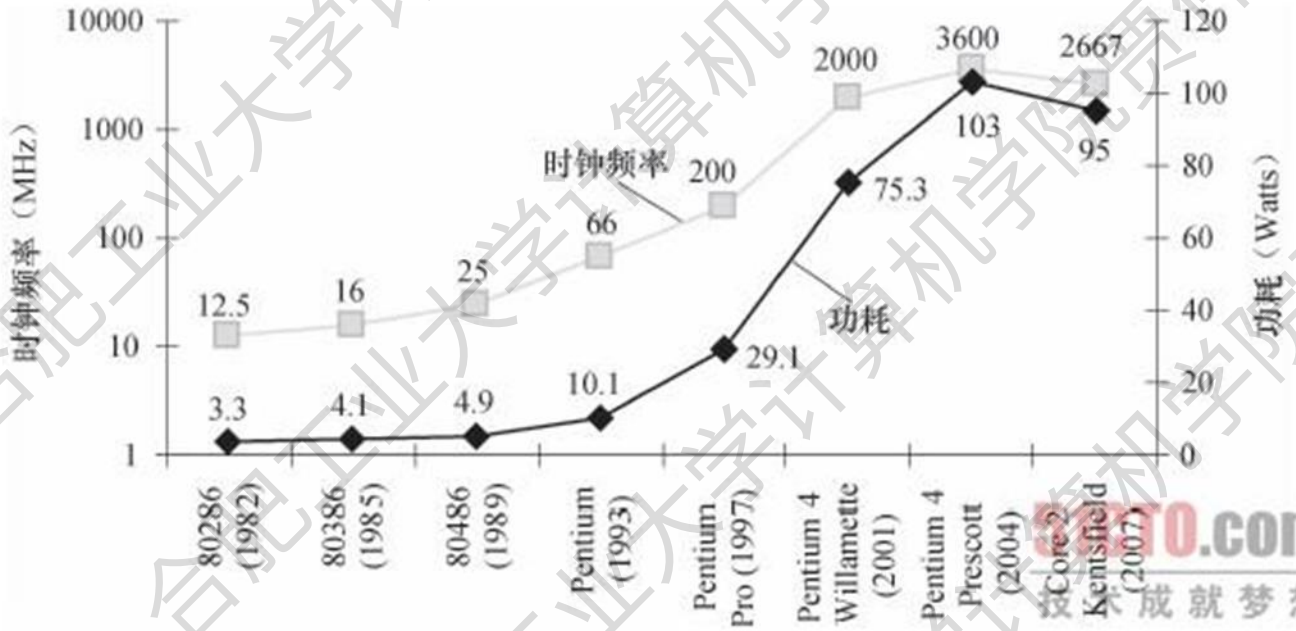
# 概念及技术指标



- **软件可移植性：指把使用在某个系列计算机中的软件直接或很少修改就能运行在另外一个系列计算机的可能性。**
  - 希望同一系列的计算机中要有软件兼容性，至少期望后来出现的性能更高的计算机要能运行早期计算机系统中已有的软件。
  - 也多指不同系列的计算机中的程序是否可以直接移植或者实现移植的难易程度，通常只有高级语言的源程序才有移植的可能性，汇编语言及机器语言基本没有移植可能性。

合肥工业大学计算机学院贾伟

# 功耗墙




**25年间Intel x86八代微处理器的时钟频率和功耗增长趋势**

- 两者的增长几乎保持了将近20年，但近几年来突然缓和下来
- 两者是密切相关的，而且功耗已经到达了极限，无法再将处理器冷却下来
- 奔腾4处理器时钟频率和功耗提高很大，但是性能提升不大。
- Prescott发热问题导致奔腾4处理器（生产线）的放弃。
- Core 2（生产线）恢复使用低时钟频率的简单流水线和片上多处理器。

# 主要性能指标

- 可靠性
  - 衡量计算机系统在应用或系统错误面前，在意外或错误使用的情况下维持软件系统的功能特性的基本能力。它反映了计算机系统在长时间运行过程中，能够稳定地提供服务而不出现故障的概率。
- 能耗
  - 衡量计算机系统在运行过程中消耗的能源。随着绿色计算和节能减排的日益重要，能耗已经成为评价计算机系统性能的一个重要指标。

合肥工业大学计算机学院贾伟

# 主要性能指标

- 可用性
  - 衡量计算机系统能够正常运行的时间比例，经常用两次故障之间的时间长度或在出现故障时系统能够恢复正常的速度来表示。它也反映了计算机系统的使用效率，即在执行任务的任意时刻能正常工作的概率。
- 可维护性
  - 衡量一个系统的可修复性和可改进性的难易程度。它反映了计算机系统在发生故障后能够迅速恢复并重新提供服务的能力，以及系统适应变化和改进的灵活性。

合肥工业大学计算机学院贾伟

# 主要性能指标

- 安全性
  - 衡量计算机在保护数据安全方面的能力，包括防止数据泄露、篡改和破坏等方面的能力。安全性是计算机系统的一个基本需求，对于保护用户隐私和信息安全至关重要。

合肥工业大学计算机学院贾伟